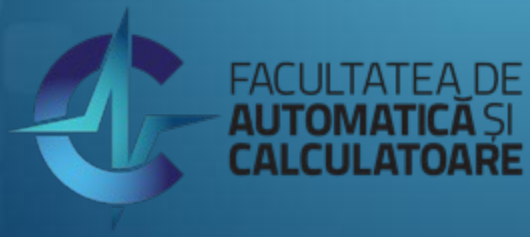


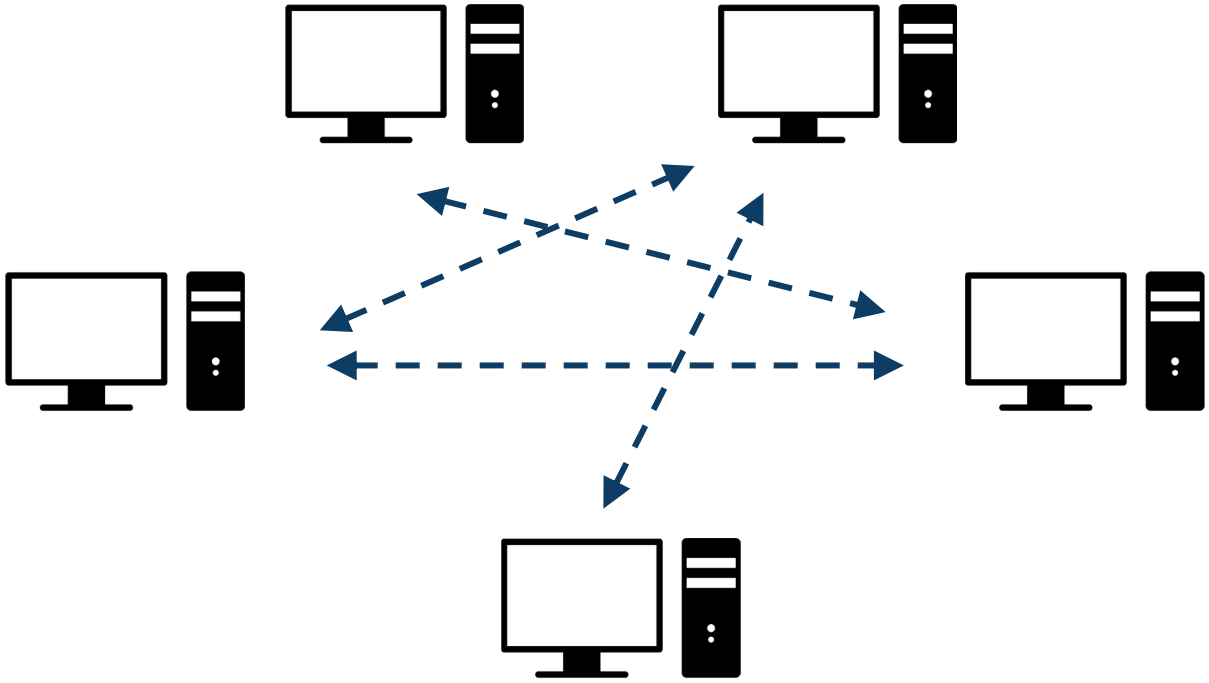

# Algoritmi Paraleli și Distribuiți
# Algoritmi peer-to-peer

**Radu-Ioan Ciobanu**
radu.ciobanu@upb.ro



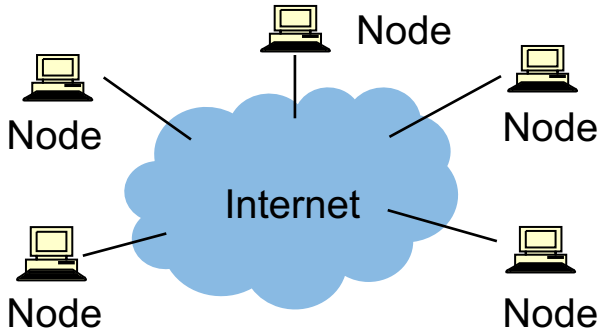



# Peer-to-peer (P2P)



# Ce este un sistem Peer-to-Peer (P2P)?

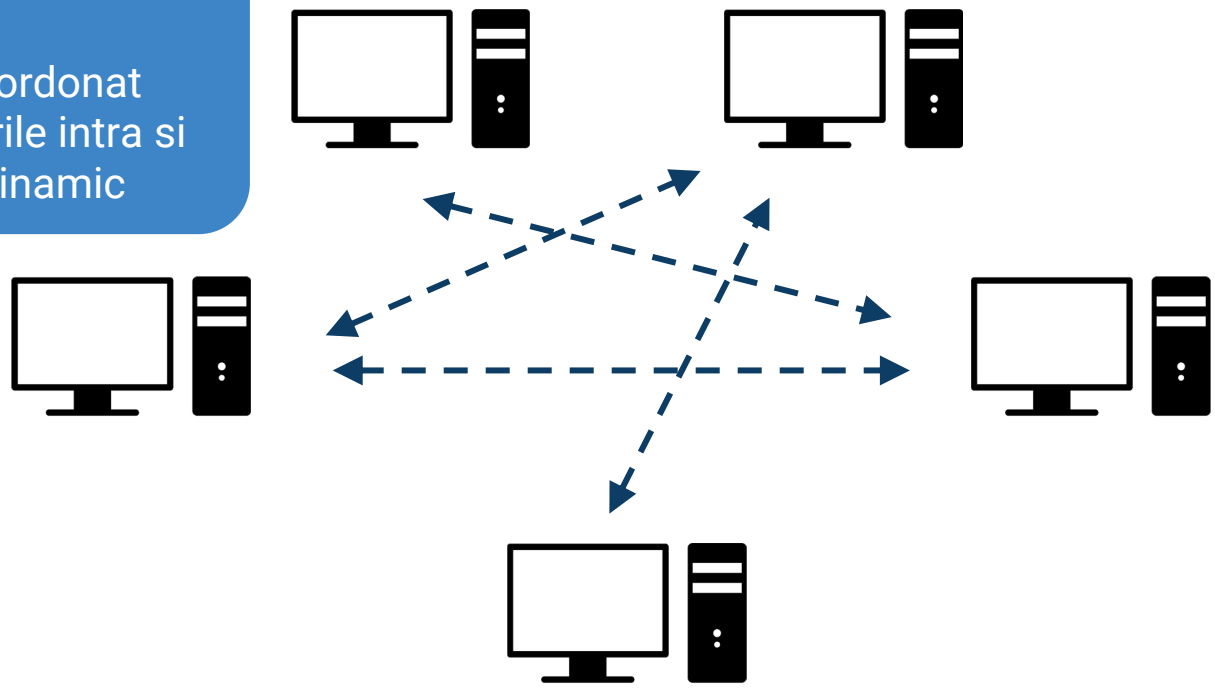
- O arhitectura de sistem distribuit in care:
  - **Nu exista un control centralizat**
  - Nodurile sunt aproape **simetrice** in functionalitate
- Numar mare de **noduri unreliable**
- **Capacitate mare de deservire a serviciilor**, prin paralismul adus de:
  - Multe discuri
  - Multe legaturi de retea
  - Multe CPI-uri
- **Absenta unui server centralizat** inseamna:
  - Mai putine sanse de overload
  - Deployment mai usor
  - Un defect nu poate strica intregul sistem
  - Sistemul per ansamblu e mai greu de atacat



# Peer-to-peer (P2P)

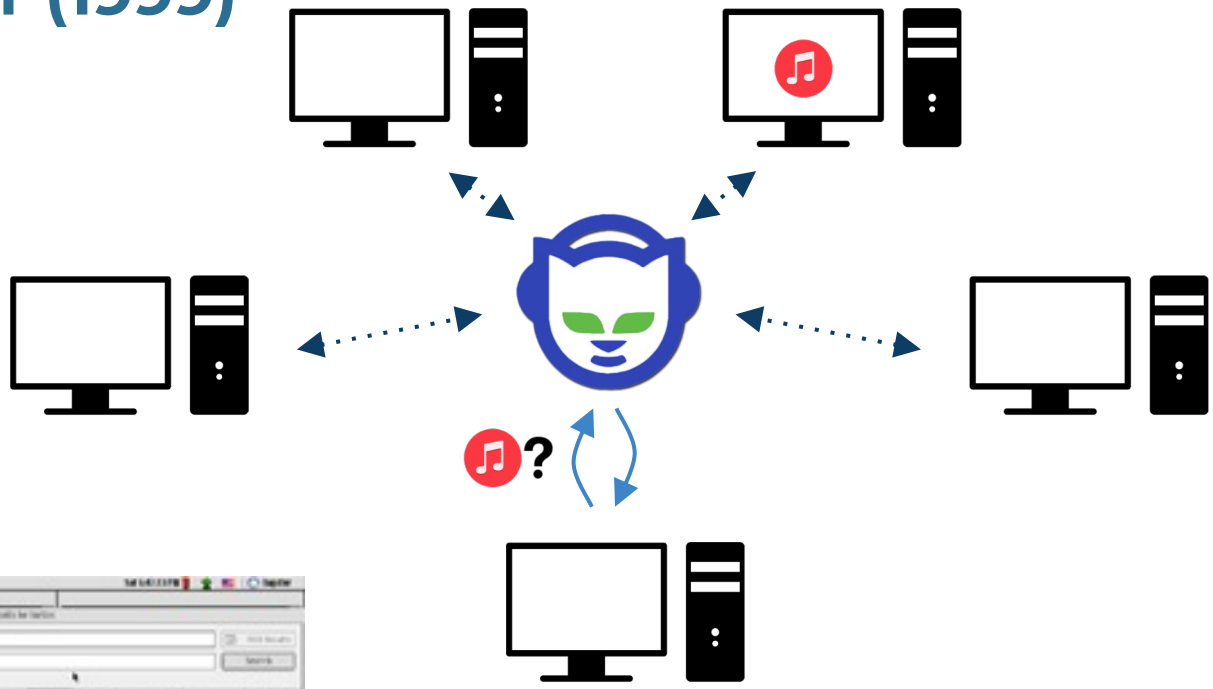
Descentralizare!

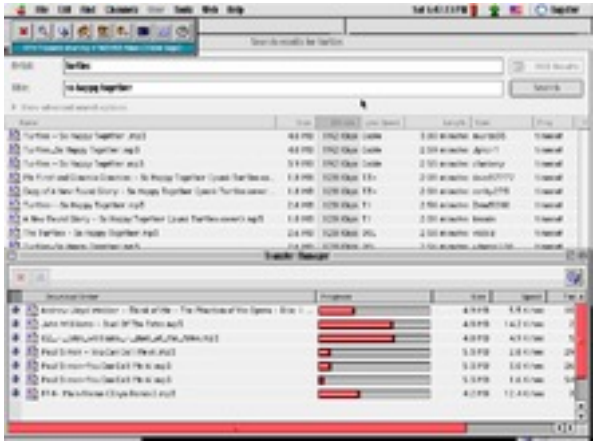
Greu de coordonat cand nodurile intra si parasesc dinamic



# Peer-to-peer (P2P)

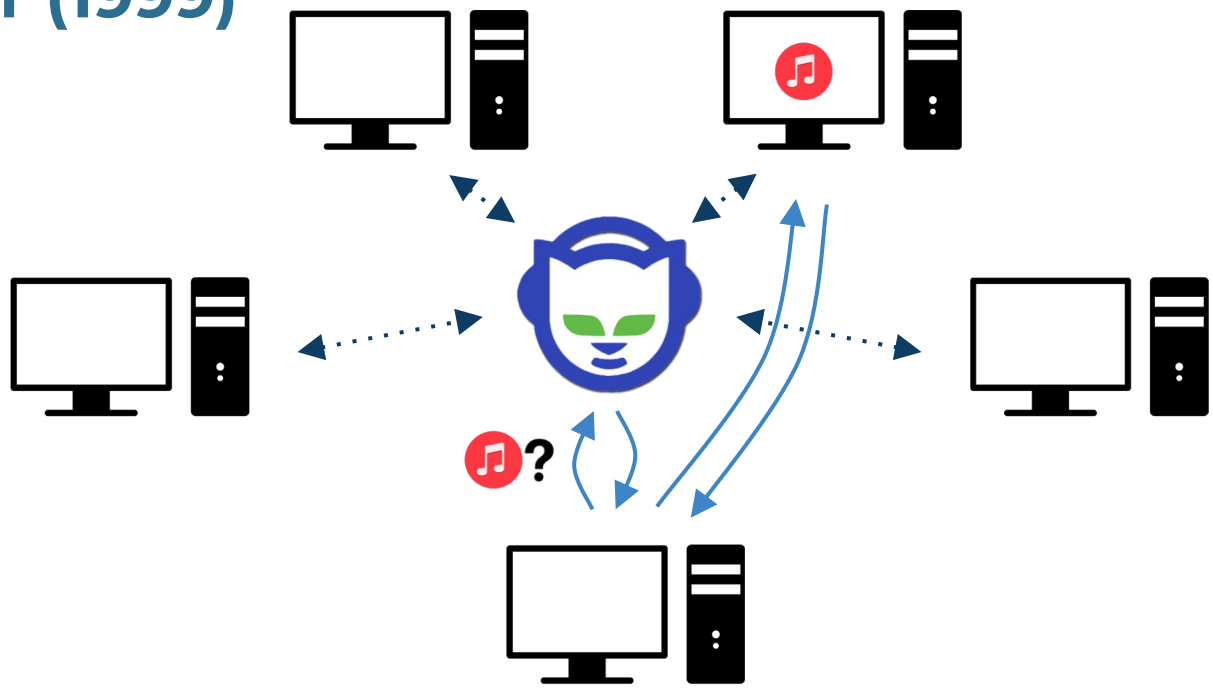
## Napster (1999)





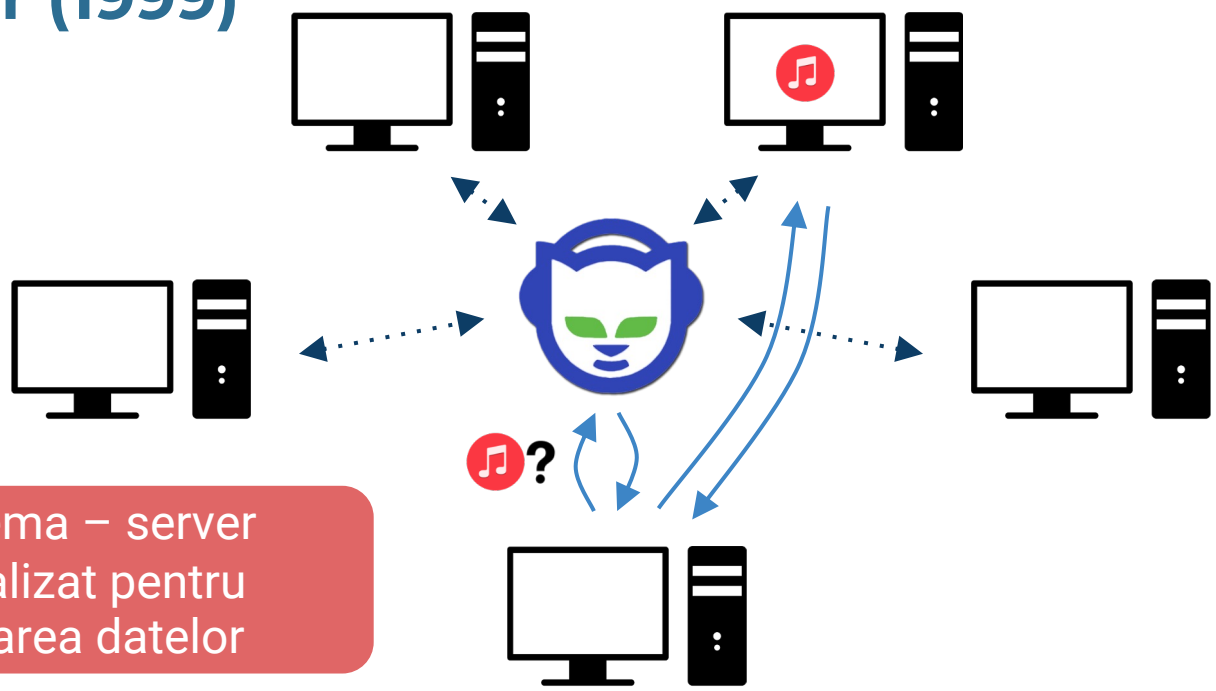
# Peer-to-peer (P2P)

## Napster (1999)



# Peer-to-peer (P2P)

## Napster (1999)



Problema – server centralizat pentru indexarea datelor

This article is more than **19 years old**

# Napster loses net music copyright case

A federal judge in San Francisco yesterday ordered Napster, the internet service that allows the trading of MP3 sound files by linking personal computers, to stop permitting the exchange of copyrighted music owned by major music labels.

The preliminary injunction, which could effectively shut down Napster's service, represents a major victory for the world's five largest record companies who are suing Napster for copyright infringement.

In granting the industry's request for a preliminary injunction, Chief Judge Marilyn Patel of the US district court in San Francisco ruled that the hugely popular Napster service, with an estimated 20m users, was used primarily to trade copyrighted material with the full knowledge of Napster.

"A majority of Napster users use the service to download and upload copyrighted music," she said.

The injunction is scheduled to take effect at midnight on Friday and will continue pending trial.

Advertisement

Spread your content, not your workflow.



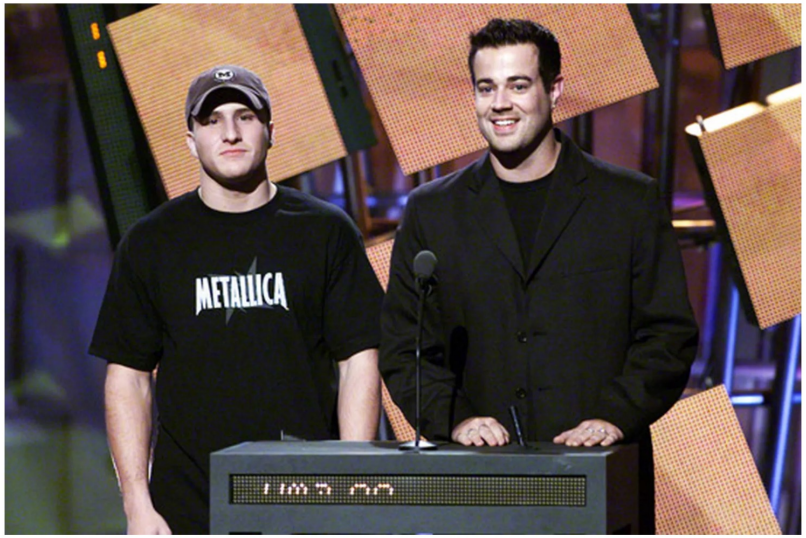


# Metallica sued Napster 15 years ago today

*File sharing forever*

By Nilay Patel | @reckless | Apr 13, 2015, 10:39am EDT

SHARE

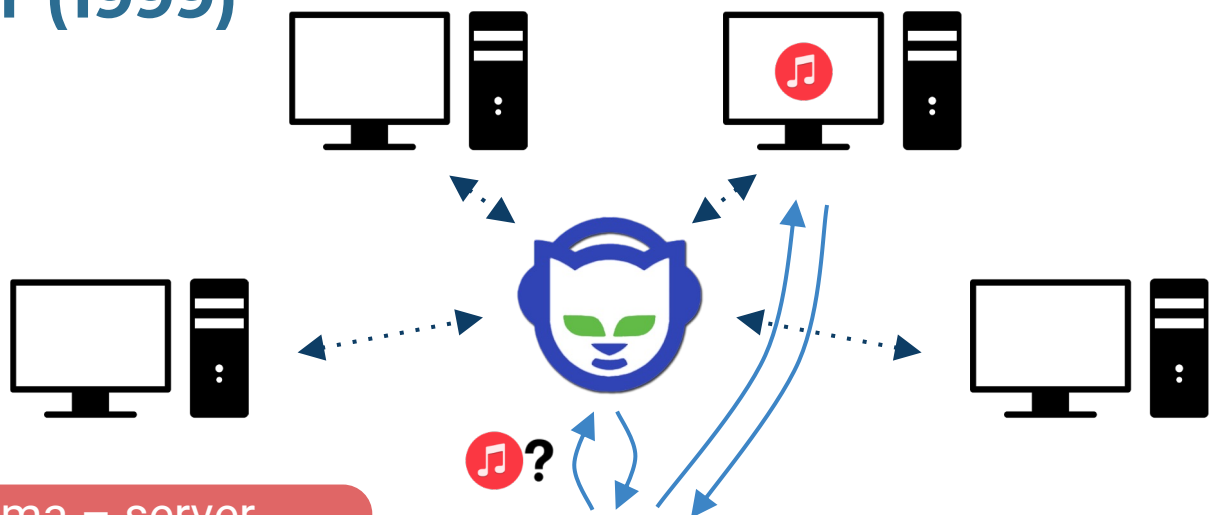



April 13th, 2000. The day the music industry and the internet became best frenemies forever.

That's the day *Metallica v. Napster, Inc.* was filed in the United States District Court for the

# Peer-to-peer (P2P)

## Napster (1999)



Problema – server centralizat pentru indexarea datelor

Solutia - Distributed hash table

gnutella

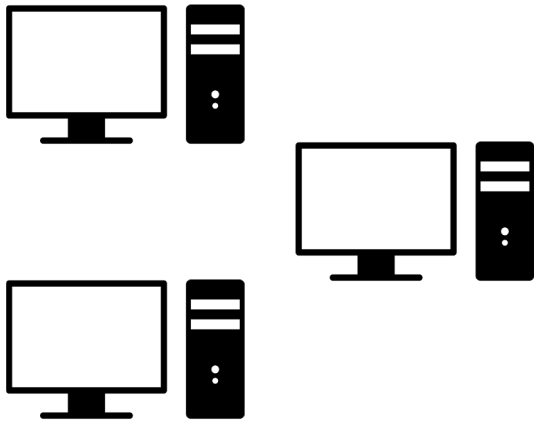
KaZaA

limewire

# Distributed Hash Tables (DHT)

Hash table

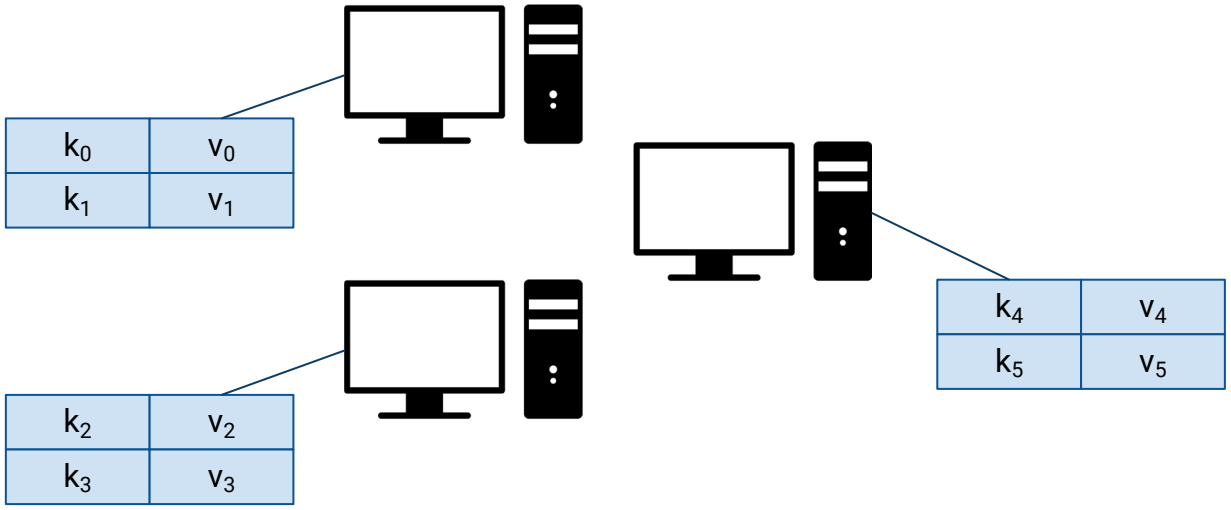
| | |
|---|---|
| $k_0$ | $v_0$ |
| $k_1$ | $v_1$ |
| $k_2$ | $v_2$ |
| $k_3$ | $v_3$ |
| $k_4$ | $v_4$ |
| $k_5$ | $v_5$ |



# Distributed Hash Tables (DHT)

Hash table

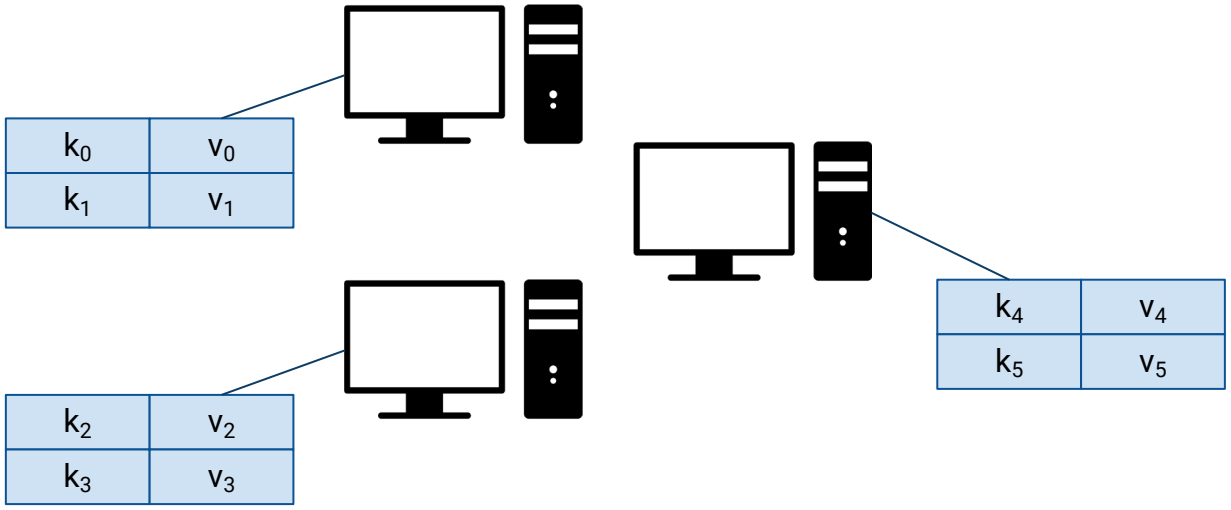
| | |
|---|---|
| $k_0$ | $v_0$ |
| $k_1$ | $v_1$ |
| $k_2$ | $v_2$ |
| $k_3$ | $v_3$ |
| $k_4$ | $v_4$ |
| $k_5$ | $v_5$ |



# Distributed Hash Tables (DHT)

Proprietati dorite:

- Descentralizare
- Load-balancing
- Scalabilitate
- Disponibilitate



# Distributed Hash Tables

- *Distributed Hash Table:*

```
key = hash(data)
lookup(key) → IP addr
send-RPC(IP address, put, key, data)
send-RPC(IP address, get, key) → data
```

(Chord lookup service)

- **Partitionam datele** peste sisteme distribuite pe scara larga
  - Punem tupluri intr-un global database engine
  - si blocurile de date intr-un sistem global de fisiere
  - peste un P2P file-sharing system

*RPC = Remote Procedure Call

# BitTorrent peste DHT

- BitTorrent poate folosi DHT in loc de (sau in conjuctie cu un) tracker

- Clientii BT folosesc DHT:
  - Key = **file content hash** (“infohash”)
  - Value = Adresa **IP a peer-ului** ce poate servi fisierul
    - Poate stoca mai multe valori (*i.e.* IP addresses) pentru o cheie

- Clientul apeleaza:
  - `get(infohash)` pentru a gasi alti clienti ce pot servi fisierul
  - `put(infohash, my-ipaddr)` pentru a se identifica ca detinator al fisierului

# Chord: un DHT performant

# Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications

Ion Stoica* Robert Morris† David Liben-Nowell†
David Karger† M. Frans Kaashoek† Frank Dabek†
Hari Balakrishnan†

January 10, 2002

**Abstract**

A fundamental problem that confronts peer-to-peer applications is to efficiently locate the node that stores a particular data item. This paper presents *Chord*, a distributed lookup protocol that addresses this problem. Chord provides support for just one operation: given a key, it maps the key onto a node. Data location can be easily implemented on top of Chord by associating a key with each data item, and storing the key/data item pair at the node to which the key maps. Chord adapts efficiently as nodes join and leave the system, and can answer queries even if the system is continuously changing. Results from theoretical analysis and simulations show that Chord is scalable, with communication cost and the state maintained by each node scaling logarithmically with the number of Chord nodes.

## 1 Introduction

Peer-to-peer systems and applications are distributed systems without any centralized control or hierarchical organization, where the software running at each node is equivalent in functionality. A review of the features of recent peer-to-peer applications yields a long list: redundant storage, permanence, selection

*University of California, Berkeley. istoica@cs.berkeley.edu
†MIT Laboratory for Computer Science, {rtm, dln, karger, kaashoek, fdabek, hari}@lcs.mit.edu.

Authors in reverse alphabetical order.



Stoica, I., Morris, R., Karger, D., Kaashoek, M. F., & Balakrishnan, H. (2001). Chord: A scalable peer-to-peer lookup service for internet applications. *ACM SIGCOMM Computer Communication Review*, *31*(4), 149-160.

# Algoritmul Chord pentru lookup

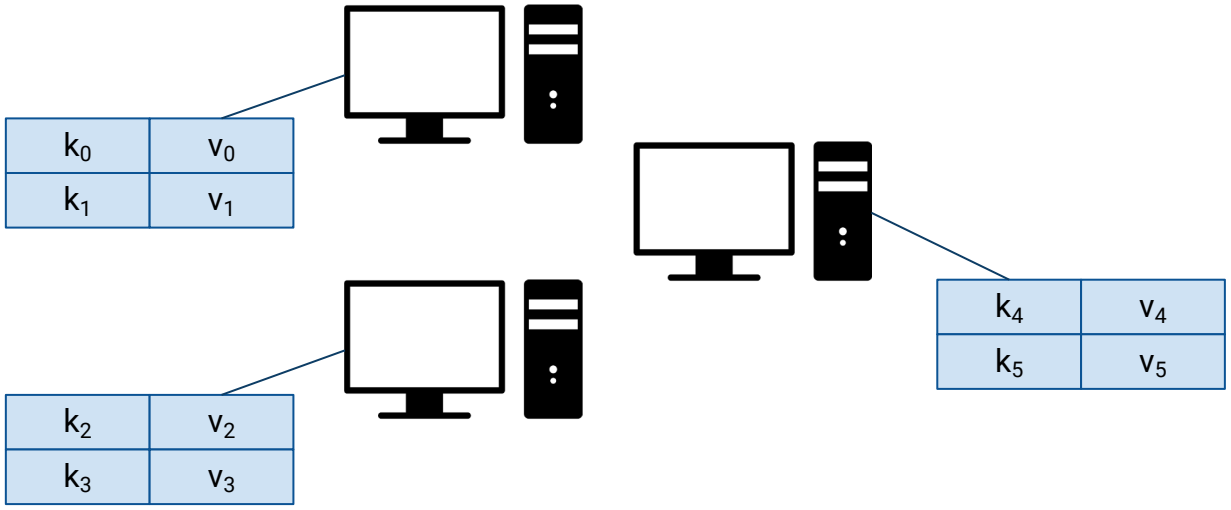
- **Interfata:** lookup(key) → IP address
- **Eficient:** O(log N) mesaje pentru operatia de lookup
  - N fiind numarul total de servere
- **Scalable:** O(log N) stari per nod
- **Robust:** poate supravietui unui numar mare de noduri defecte

# Chord – DHT eficient

Cum asignam chei nodurilor peer?

Hash table

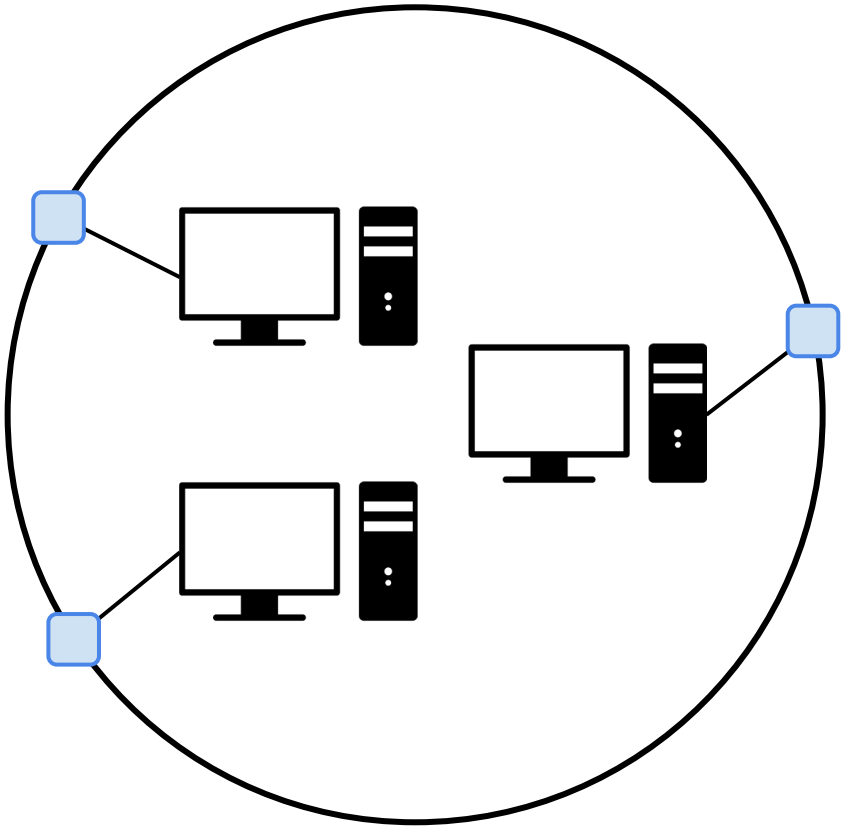
| | |
|---|---|
| $k_0$ | $v_0$ |
| $k_1$ | $v_1$ |
| $k_2$ | $v_2$ |
| $k_3$ | $v_3$ |
| $k_4$ | $v_4$ |
| $k_5$ | $v_5$ |



# Chord

Hash table

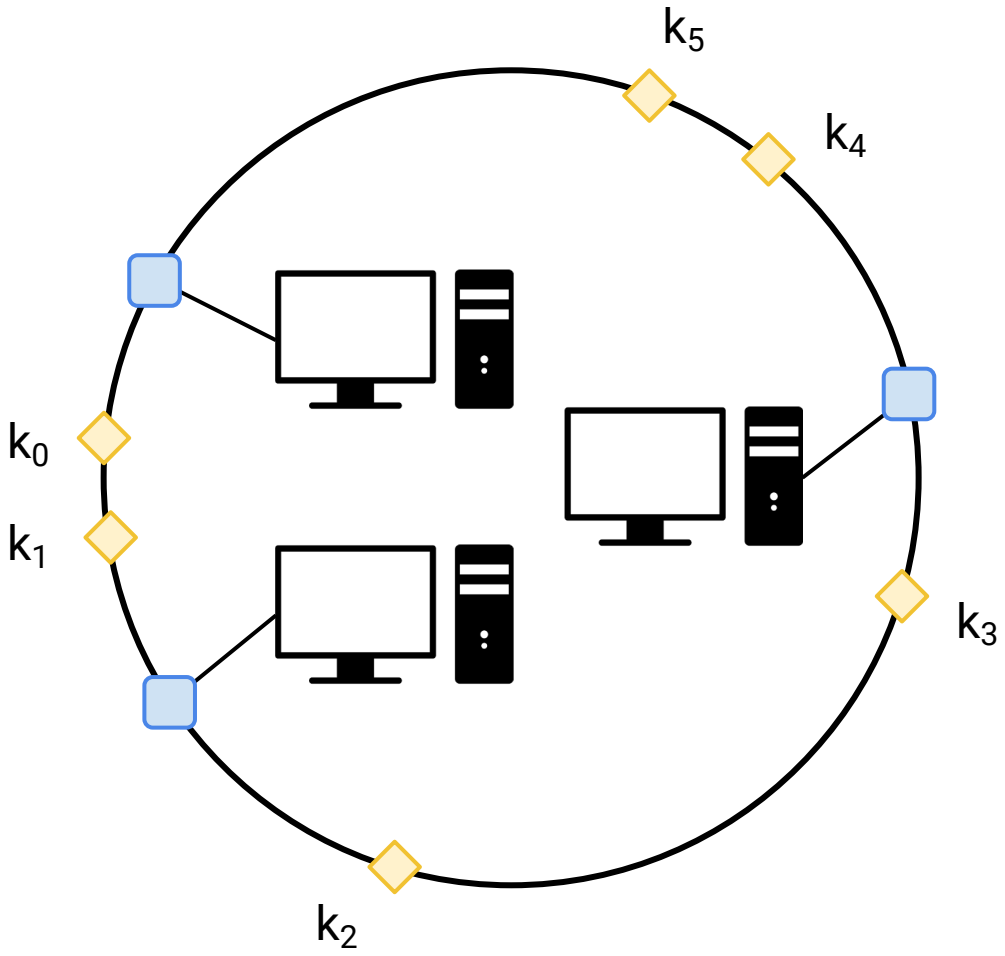
| | |
|---|---|
| $k_0$ | $v_0$ |
| $k_1$ | $v_1$ |
| $k_2$ | $v_2$ |
| $k_3$ | $v_3$ |
| $k_4$ | $v_4$ |
| $k_5$ | $v_5$ |



# Chord

Hash table

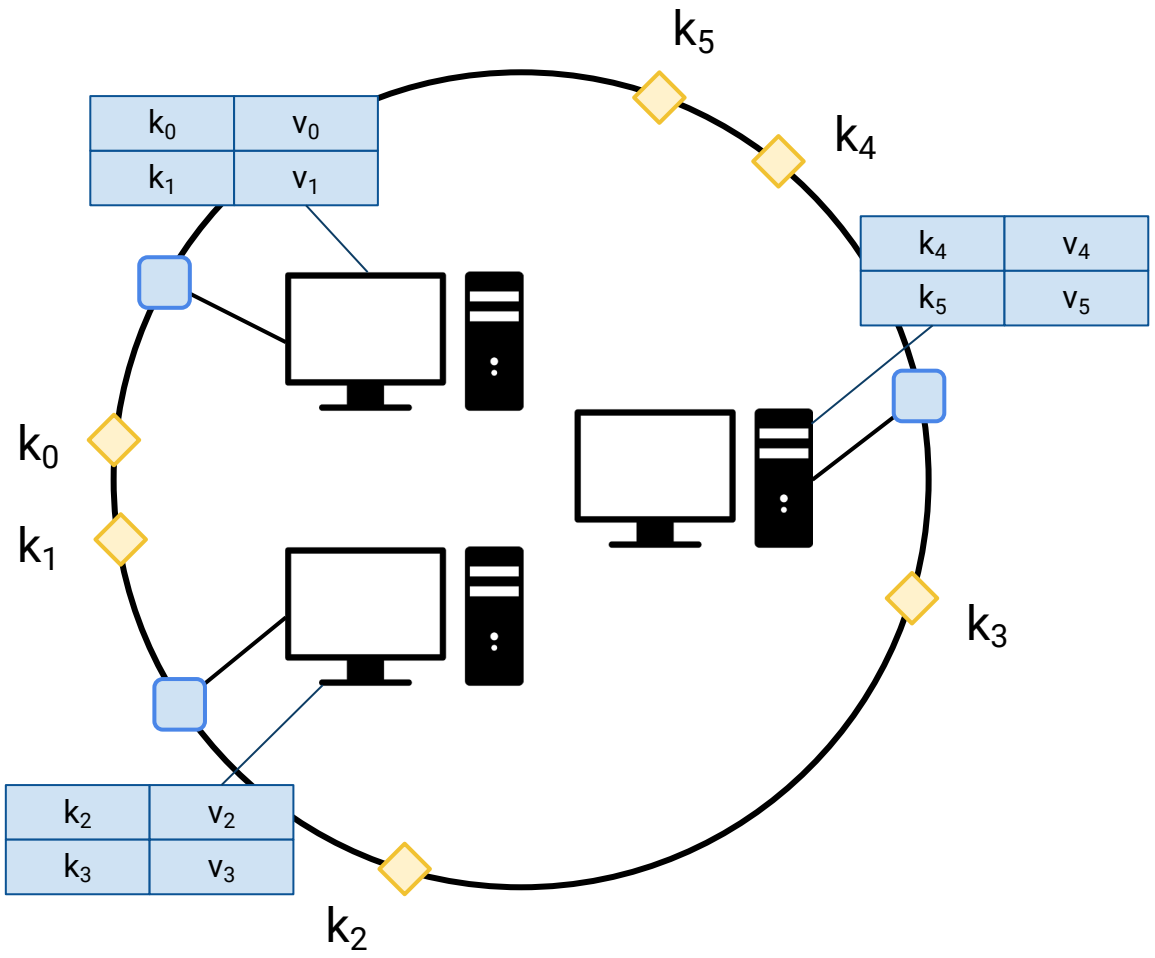
| | |
|---|---|
| $k_0$ | $v_0$ |
| $k_1$ | $v_1$ |
| $k_2$ | $v_2$ |
| $k_3$ | $v_3$ |
| $k_4$ | $v_4$ |
| $k_5$ | $v_5$ |



# Chord

Hash table

| | |
|---|---|
| $k_0$ | $v_0$ |
| $k_1$ | $v_1$ |
| $k_2$ | $v_2$ |
| $k_3$ | $v_3$ |
| $k_4$ | $v_4$ |
| $k_5$ | $v_5$ |

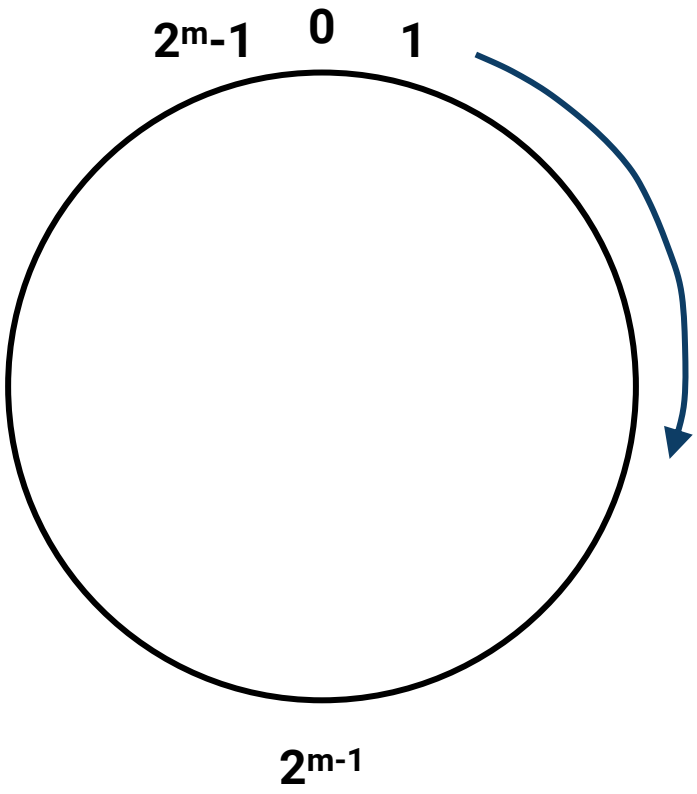


# Identificatori Chord

- **Key identifier** = SHA-1(key)
- **Node identifier** = SHA-1(IP address)
- SHA-1 asigura o distributie uniforma a ambelor
- ***Cum partitioneaza Chord datele?***
  - *i.e.*, map key IDs to node IDs

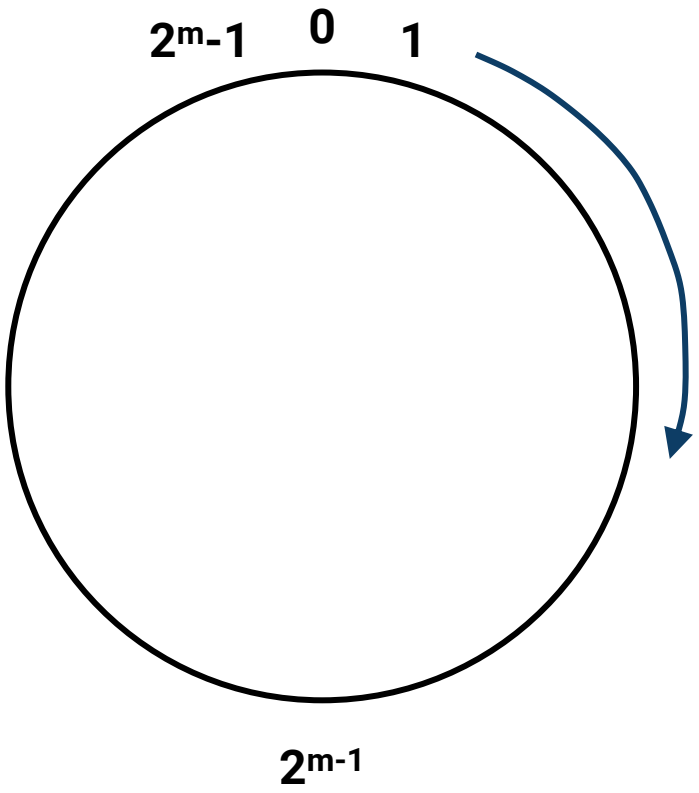
# Chord (distributia cheilor)

Inel de identificatori peste spatiul hash $2^m$



# Chord (distributia cheilor)

Inel de identificatori peste spatiul hash $2^m$

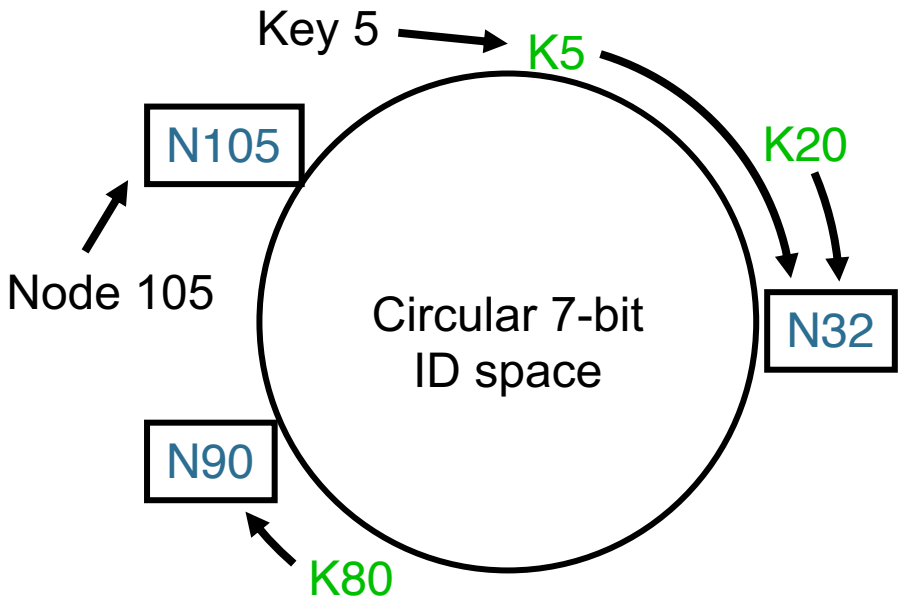



= node

= key

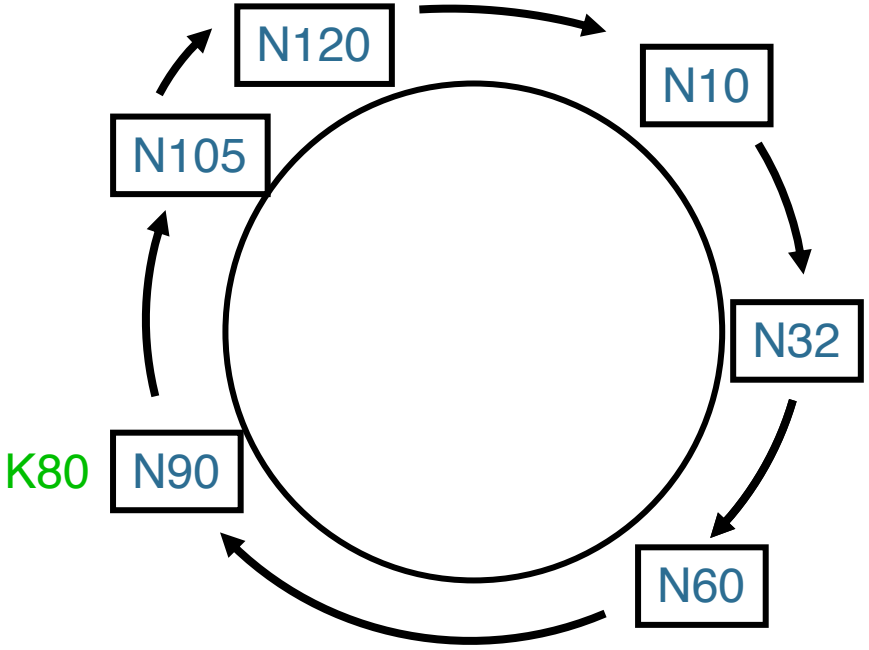
node id = hash( node )
key id = hash( key )

# Consistent hashing [Karger '97]

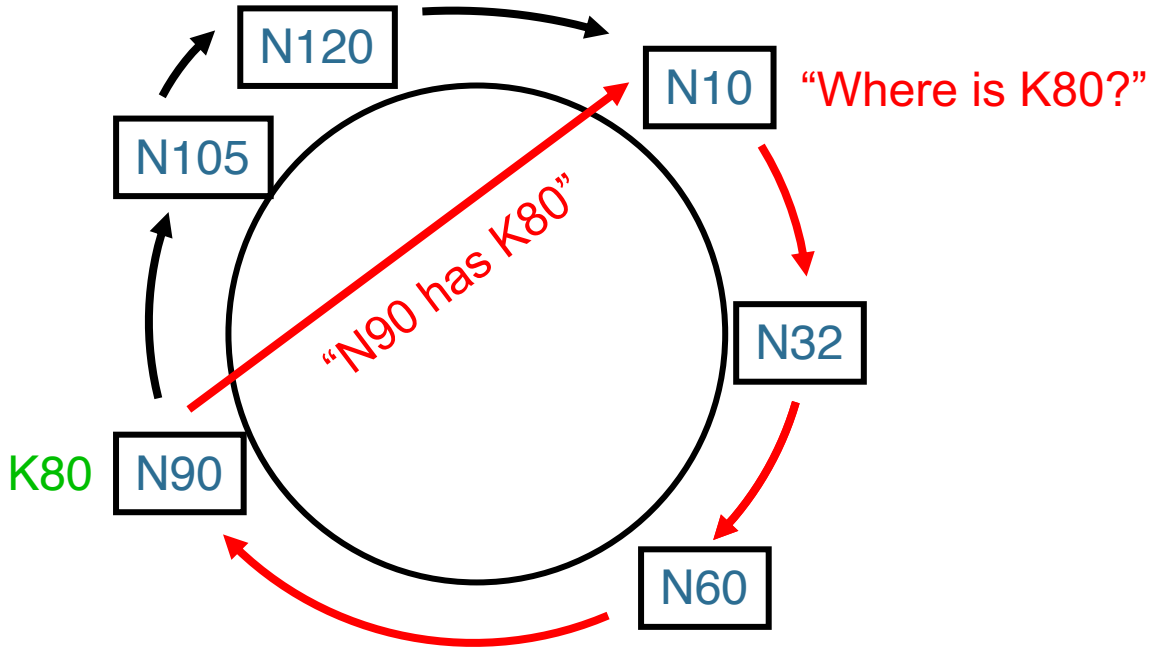



Cheia este stocata la successor: nodul avand urmatorul cel mai mare ID

# Chord: Successor pointers



# Basic lookup



# Simple lookup algorithm

```
Lookup(key-id)
  succ ← my successor
  if my-id < succ < key-id //next hop
        call Lookup(key-id) on succ
  else                                    //done
    return succ
```

- **Corectitudinea** depinde doar de **succesori**

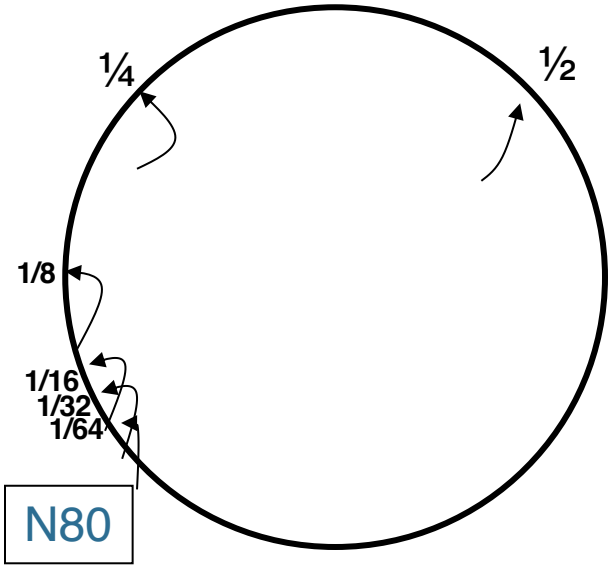
# Imbunatatirea performantei

- **Problema:** Forwardarea prin intermediul succesorului poate fi inceata

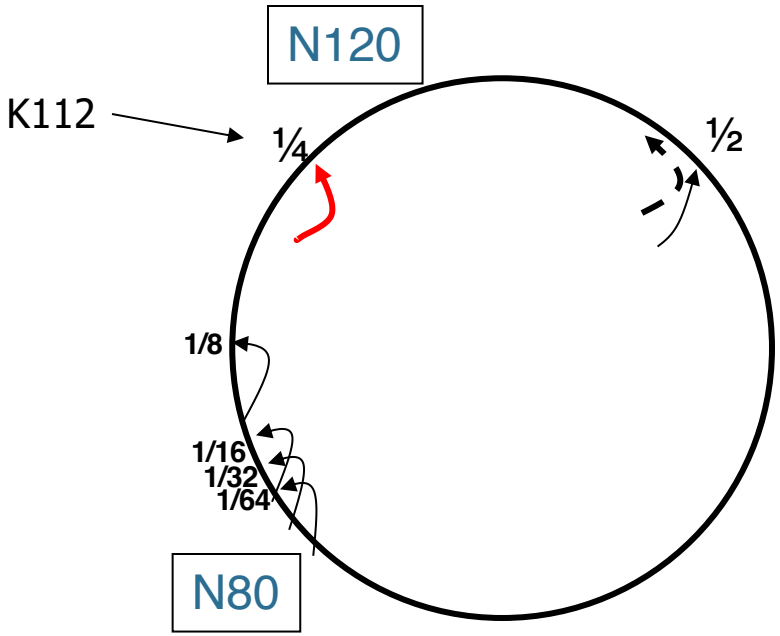
- Stuctura de date este o lista inlantuita: O(n)

- **Idee:** Putem sa facem operatia sa arate mai mult a **binary search**?
  - Adica sa imbunatatim distanta cu fiecare pas

# “Finger table” permite operatii de lookup in timp log N



# Finger $i$ redirecteaza catre succesorul lui $n+2^i$



# Implicatii ale tabelei finger

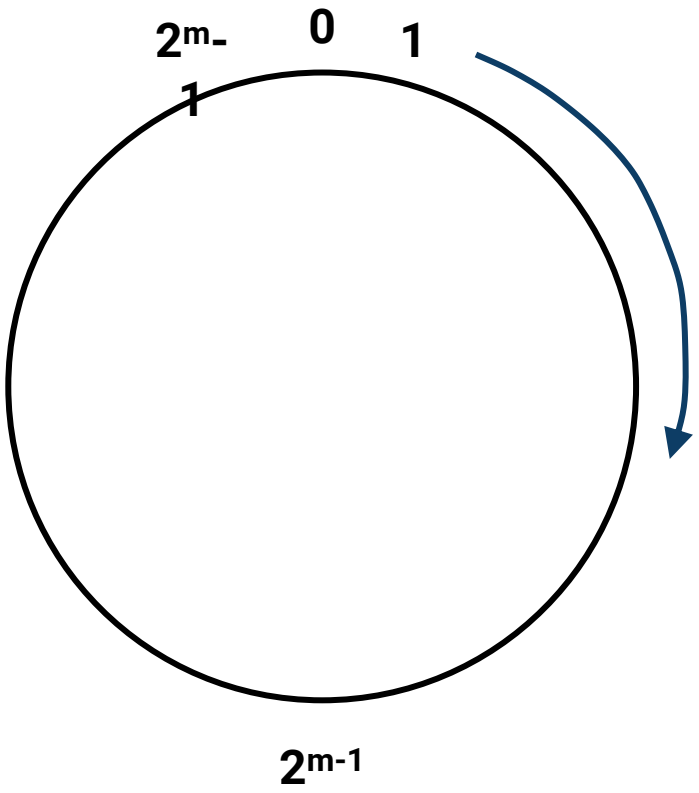
- Avem un **binary lookup tree** cu fiecare nod
  - Cu ramuri redirectate catre alte tabele finger ale celorlalte noduri
- Este mai bine decat sa aranjam nodurile intr-un singur arbore
  - Fiecare nod actioneaza ca radacina a unui arbore
    - Deci nu exista niciun **root hotspot**
    - Nu exista **single point of failure**

# Lookup with finger table

```
Lookup(key-id)
  look in local finger table for
        highest n: my-id < n < key-id
  if n exists
        call Lookup(key-id) on node n //next hop
  else
        return my successor    //done
```

# Chord

Inel de identificatori peste spatiul hash $2^m$




= node

= key

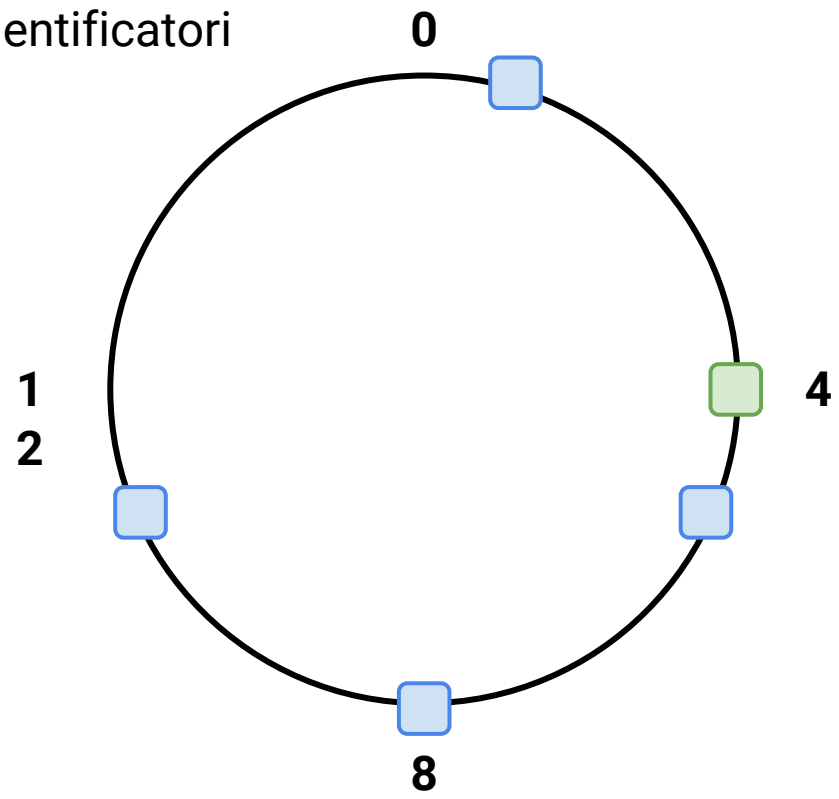
node id = hash( node )
key id = hash( key )

successor(id)
finger table for node
at id *i*

| finger | node id |
|---|---|
| 1 | succ(*i*) |
| 2 | succ(*i* + 2) |
| *j* | succ(*i* + $2^{j-1}$) |

# Chord

Inelul de identificatori

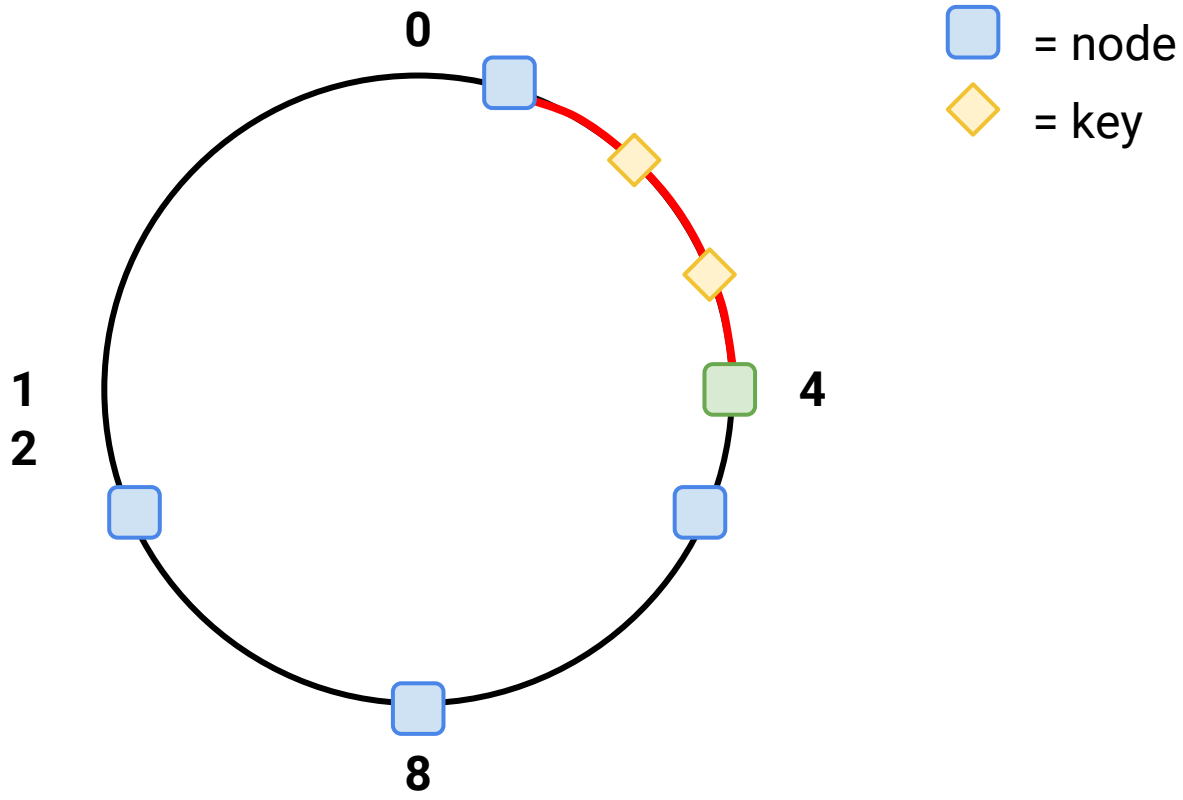



= node

= key

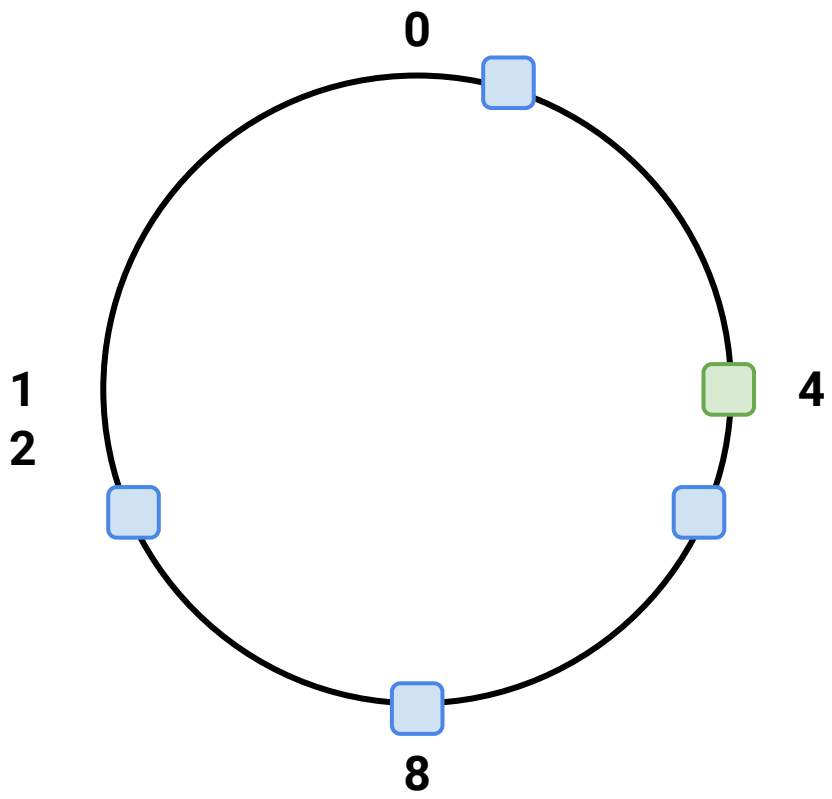
# Chord

Inel de identificatori



# Chord

Inel de identificatori



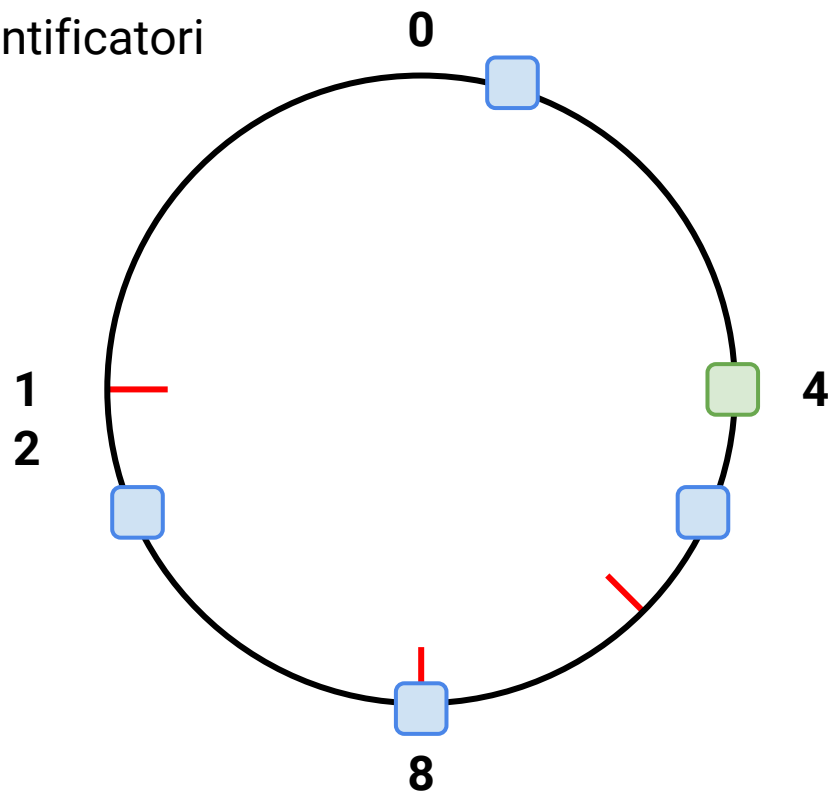

= node

= key

| finger | node id |
|---|---|
| 1 | succ($i$) |
| 2 | succ($i + 2$) |
| 3 | succ($i + 2^2$) |
| 4 | succ($i + 2^3$) |

# Chord

Inel de identificatori



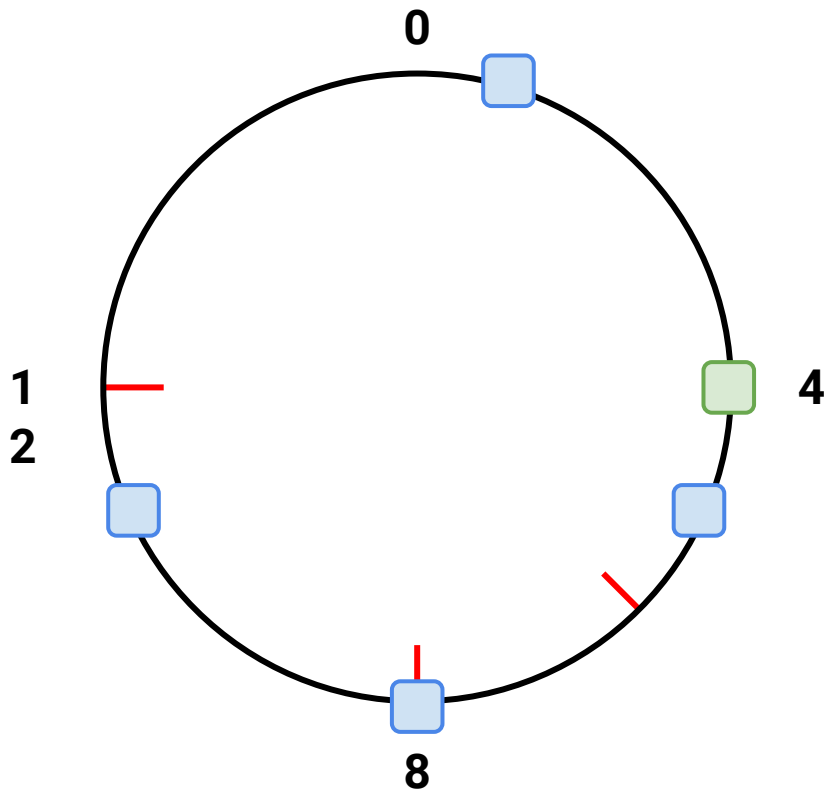

= node

= key

| finger | node id |
|---|---|
| 1 | succ(4) |
| 2 | succ(4 + 2) |
| 3 | succ(4 + $2^2$) |
| 4 | succ(4 + $2^3$) |

# Chord

Inel de identificatori




= node

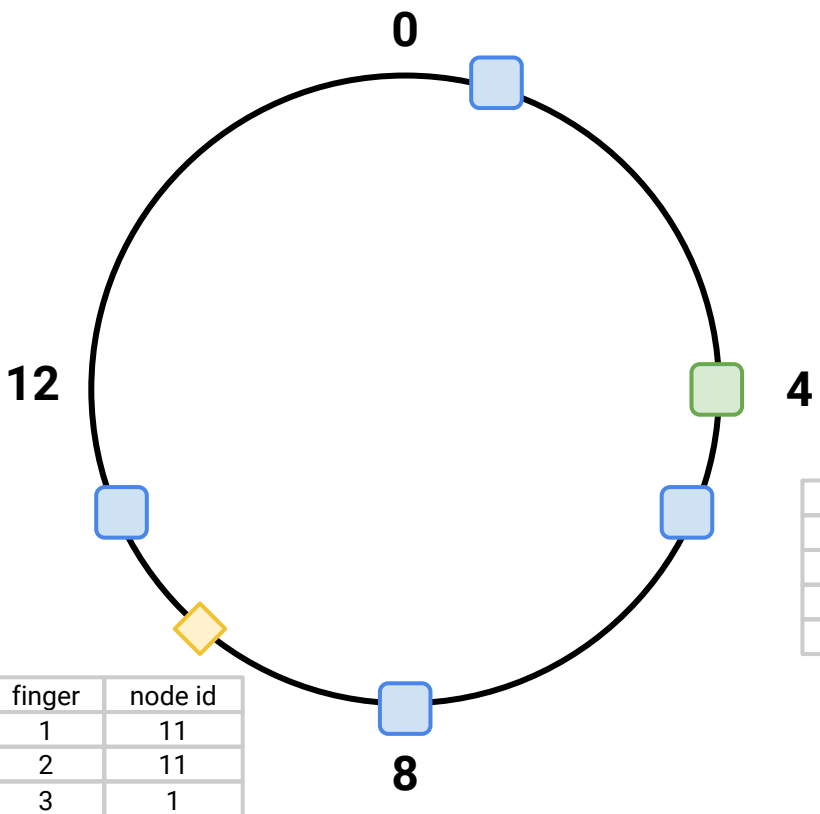
= key

| finger | node id |
|---|---|
| 1 | 5 |
| 2 | 8 |
| 3 | 8 |
| 4 | 1 |

# Asadar... Chord Lookup

```
find_successor(id):
  p = find_predecessor(id)
  return p.successor

find_predecessor(id):
  n = self
  while id not between (n, n.successor]:
    n = n.closest_preceding_finger(id)
  return n
```




| finger | node id |
|---|---|
| 1 | 11 |
| 2 | 11 |
| 3 | 1 |
| 4 | 1 |

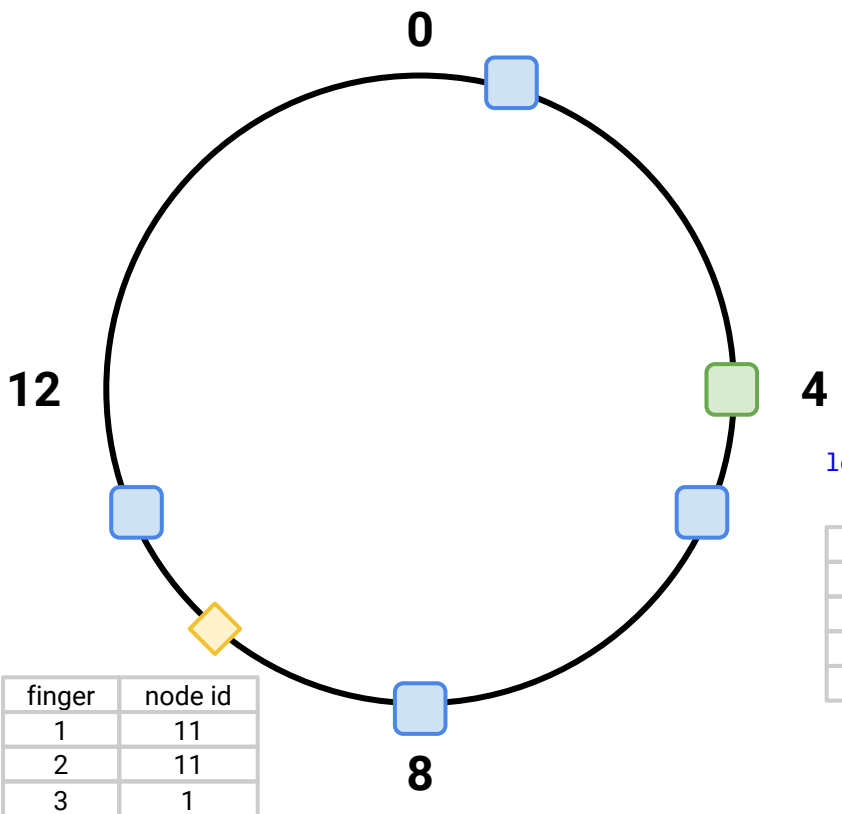
| finger | node id |
|---|---|
| 1 | 5 |
| 2 | 8 |
| 3 | 8 |
| 4 | 1 |

# Chord - Lookup

```
find_successor(id):
  p = find_predecessor(id)
  return p.successor

find_predecessor(id):
  n = self
  while id not between (n, n.successor]:
    n = n.closest_preceding_finger(id)
  return n
```

Inel de identificatori




| finger | node id |
|---|---|
| 1 | 11 |
| 2 | 11 |
| 3 | 1 |
| 4 | 1 |

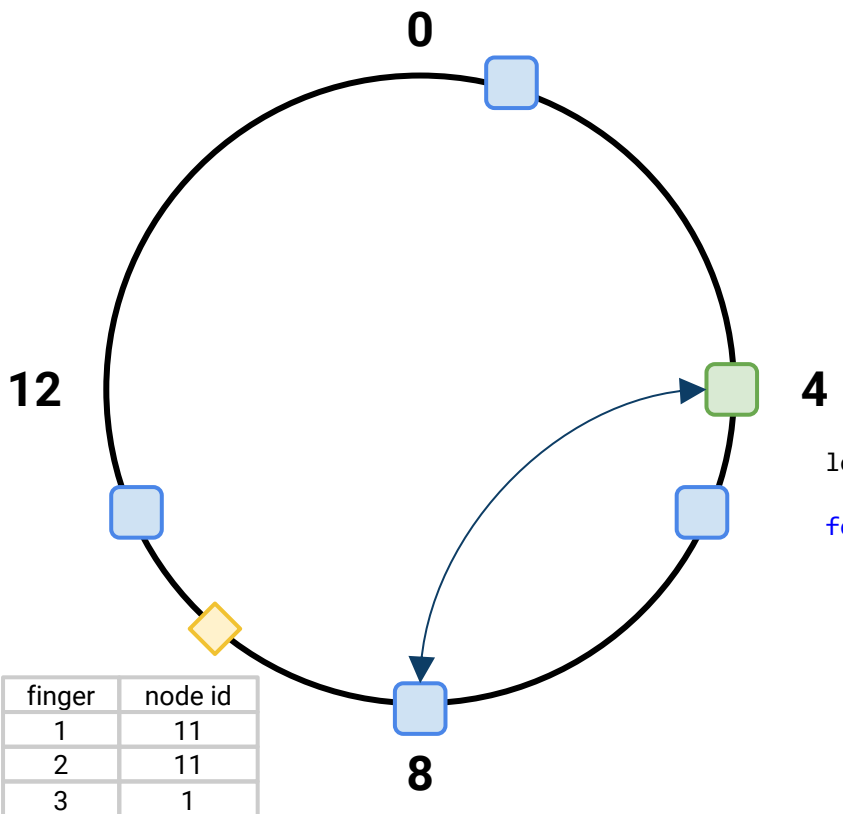
lookup(10)

| finger | node id |
|---|---|
| 1 | 5 |
| 2 | 8 |
| 3 | 8 |
| 4 | 1 |

# Chord - Lookup

```
find_successor(id):
  p = find_predecessor(id)
  return p.successor

find_predecessor(id):
  n = self
  while id not between (n, n.successor]:
    n = n.closest_preceding_finger(id)
  return n
```




| finger | node id |
|---|---|
| 1 | 5 |
| 2 | 8 |
| 3 | 8 |
| 4 | 1 |

| finger | node id |
|---|---|
| 1 | 11 |
| 2 | 11 |
| 3 | 1 |
| 4 | 1 |

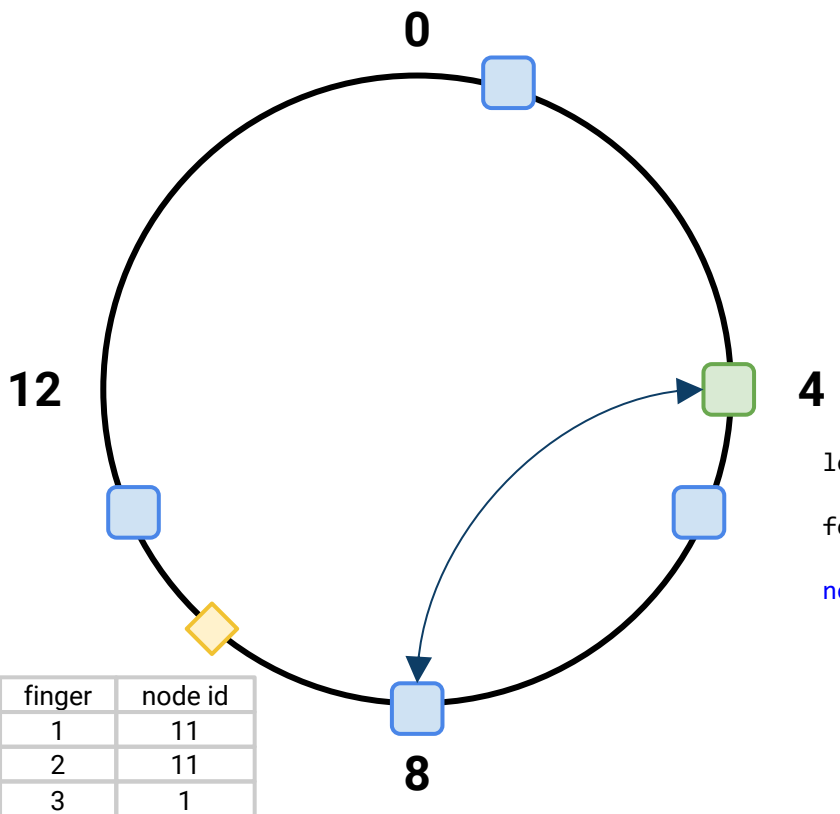
`lookup(10)`

follow finger 3 to node id 8

# Chord - Lookup

```
find_successor(id):
  p = find_predecessor(id)
  return p.successor

find_predecessor(id):
  n = self
  while id not between (n, n.successor]:
    n = n.closest_preceding_finger(id)
  return n
```




| finger | node id |
|---|---|
| 1 | 5 |
| 2 | 8 |
| 3 | 8 |
| 4 | 1 |

| finger | node id |
|---|---|
| 1 | 11 |
| 2 | 11 |
| 3 | 1 |
| 4 | 1 |

```
lookup(10)

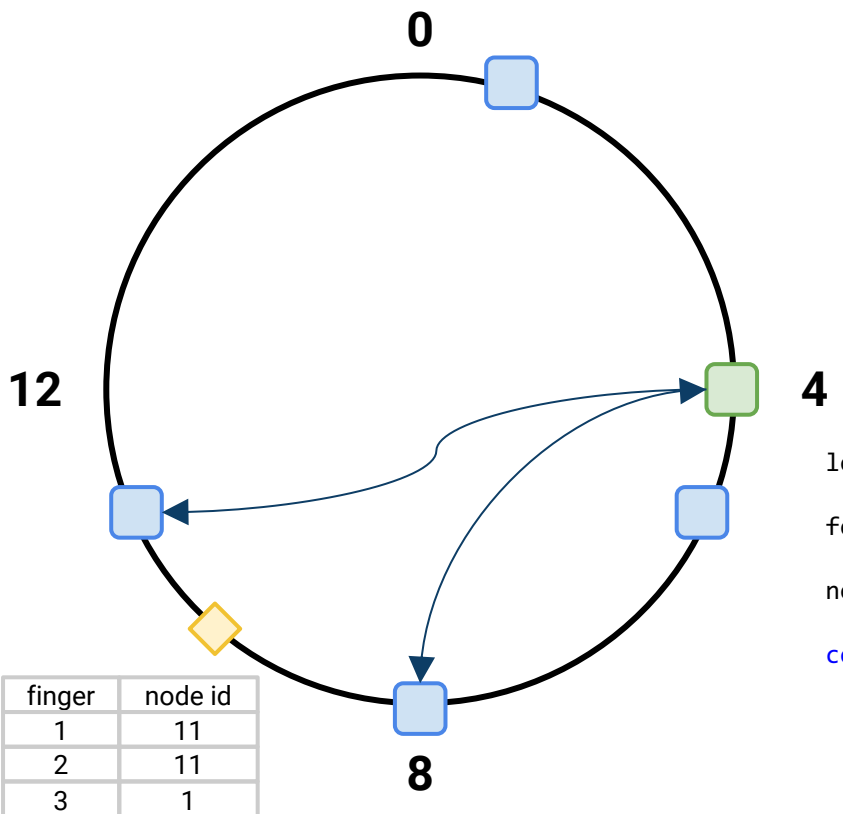
follow finger 3 to node id 8

node id 8 identifies as predecessor of id 10
```

# Chord - Lookup

```
find_successor(id):
  p = find_predecessor(id)
  return p.successor

find_predecessor(id):
  n = self
  while id not between (n, n.successor]:
    n = n.closest_preceding_finger(id)
  return n
```




| finger | node id |
|---|---|
| 1 | 5 |
| 2 | 8 |
| 3 | 8 |
| 4 | 1 |

| finger | node id |
|---|---|
| 1 | 11 |
| 2 | 11 |
| 3 | 1 |
| 4 | 1 |

lookup(10)

follow finger 3 to node id 8

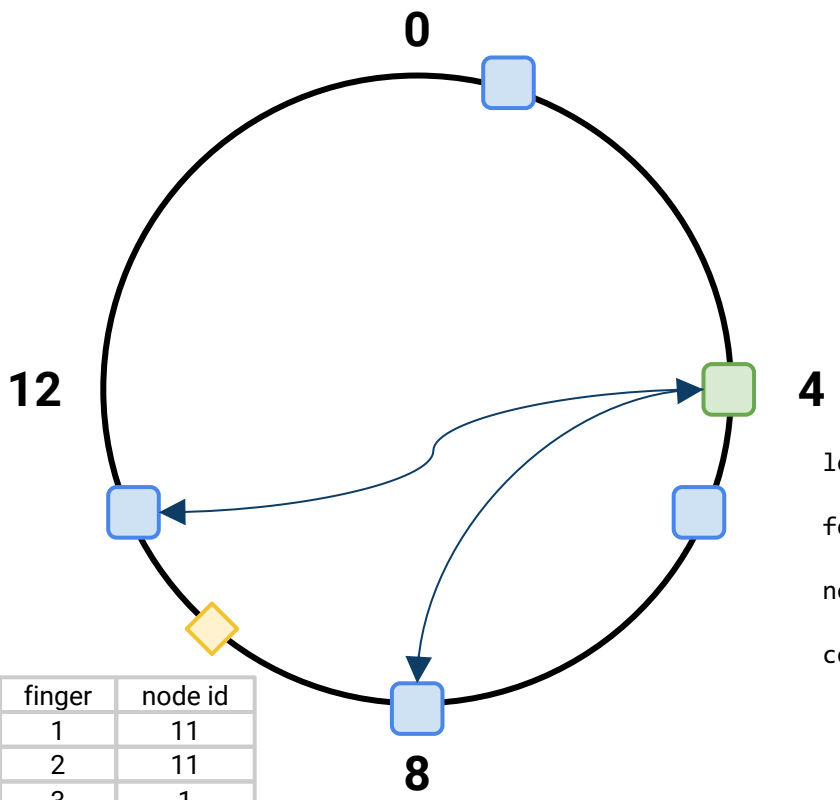
node id 8 identifies as predecessor of id 10

complete lookup at successor of node id 8

# Chord - Lookup

```
find_successor(id):
  p = find_predecessor(id)
  return p.successor

find_predecessor(id):
  n = self
  while id not between (n, n.successor]:
    n = n.closest_preceding_finger(id)
  return n
```




| finger | node id |
|---|---|
| 1 | 11 |
| 2 | 11 |
| 3 | 1 |
| 4 | 1 |

| finger | node id |
|---|---|
| 1 | 5 |
| 2 | 8 |
| 3 | 8 |
| 4 | 1 |

Nr. de hopuri?
Fiecare op. finger lookup injumatateste distanta la cheie =>
O(log N)

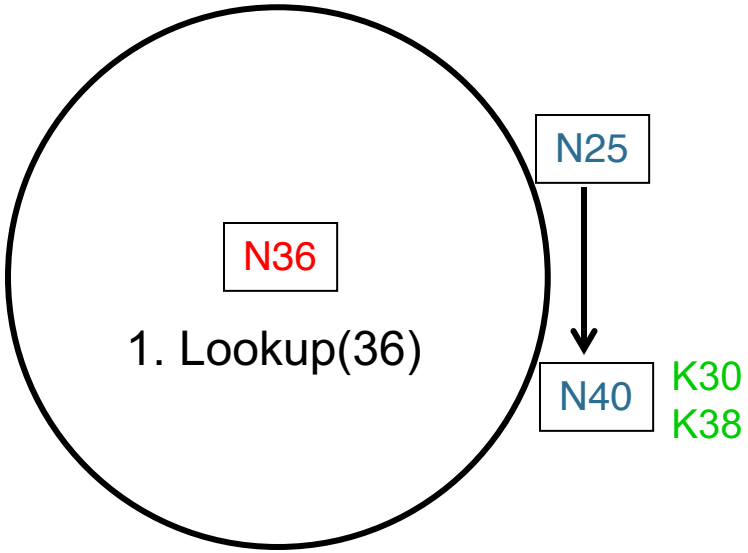
```
lookup(10)

follow finger 3 to node id 8

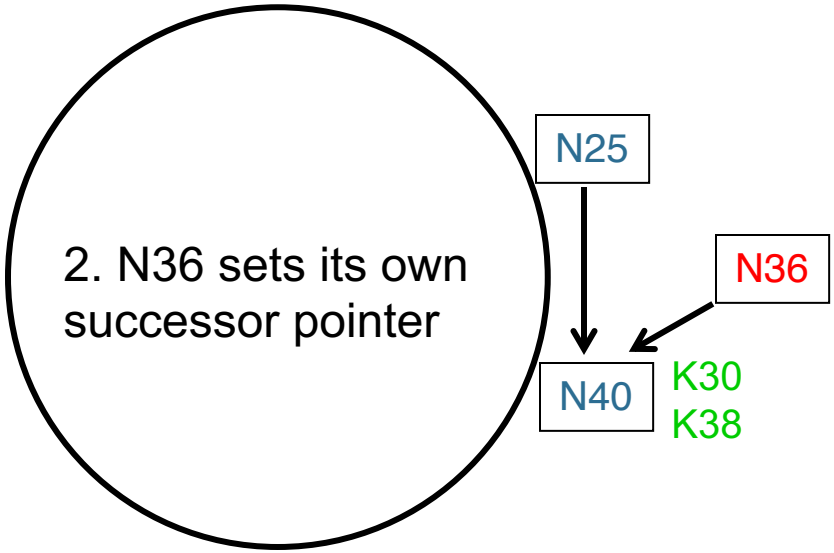
node id 8 identifies as predecessor of id 10

complete lookup at successor of node id 8
```

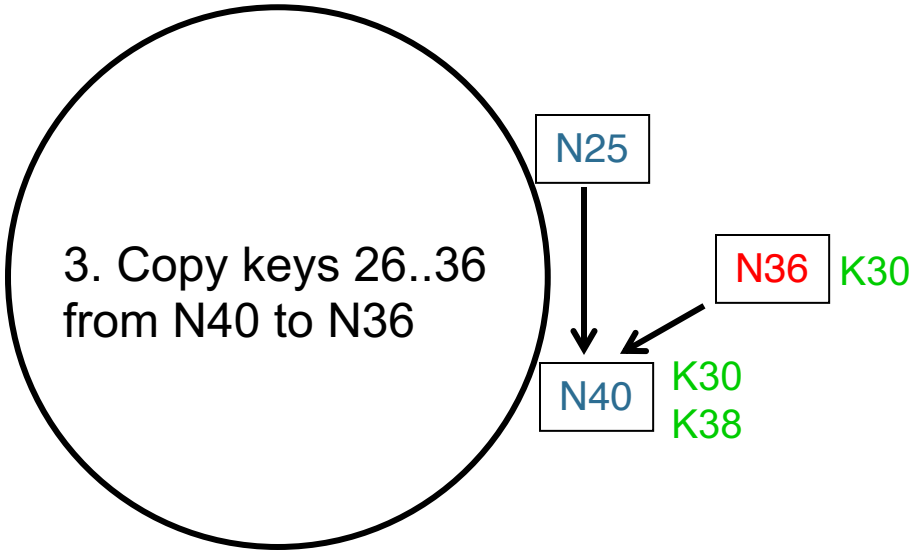
# Joining: Linked list insert



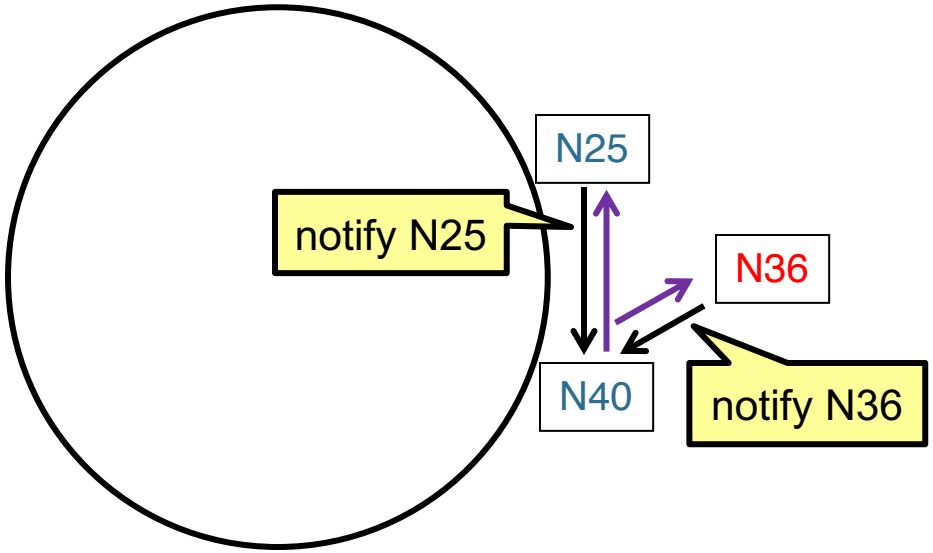
# Join (2)



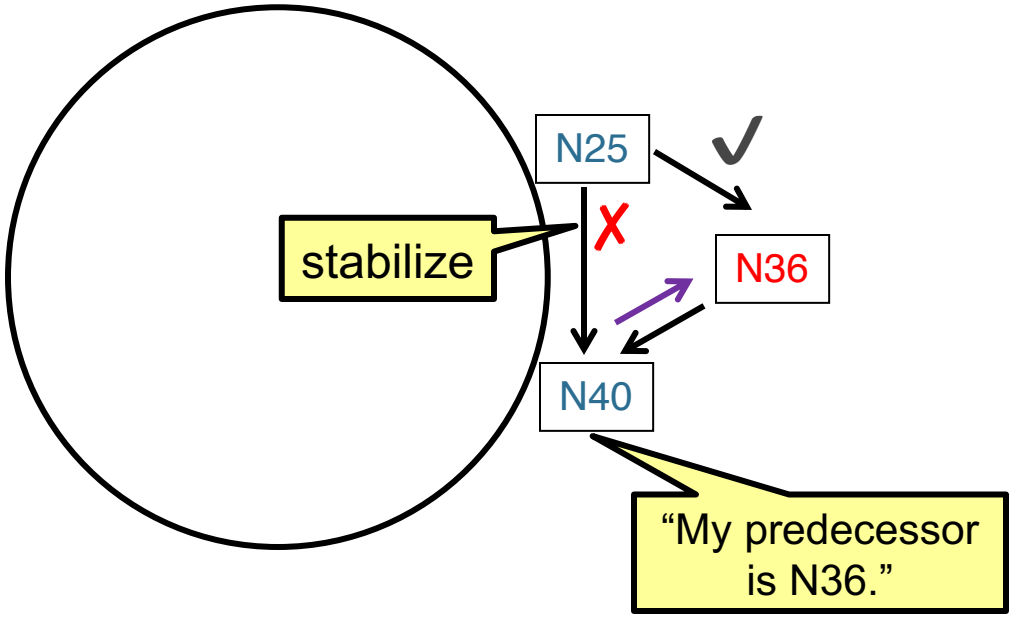
# Join (3)



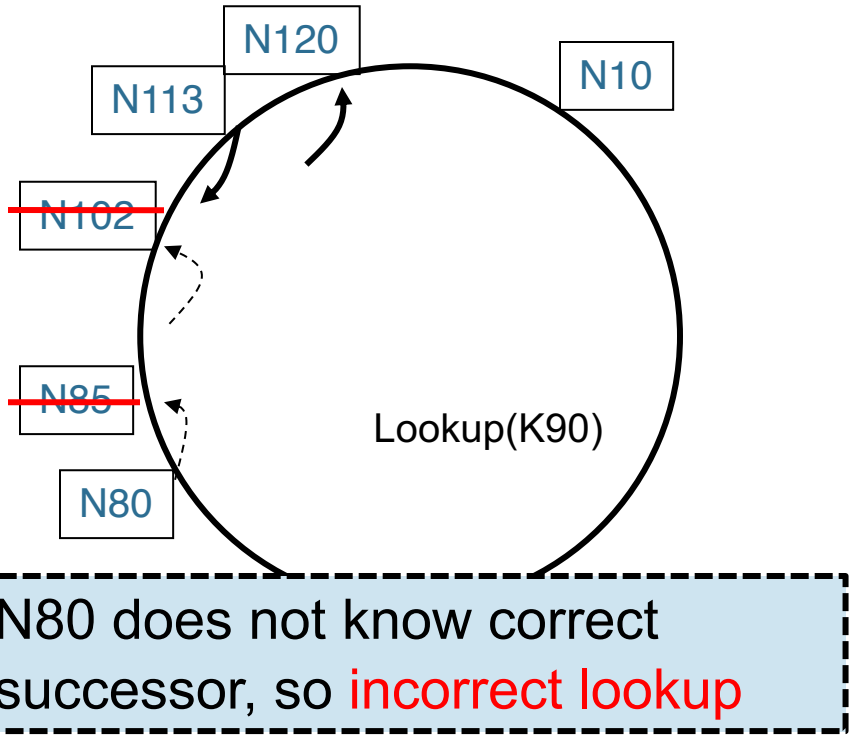
# Mesajele ***notify*** mentin predecesorii



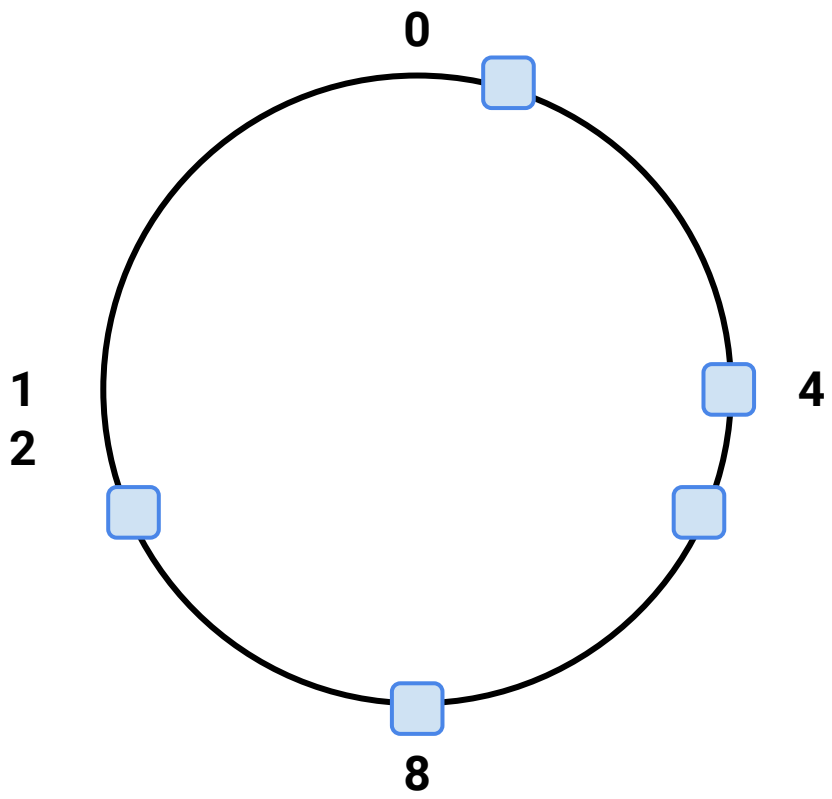
# Mesajele ***stabilize*** fixeaza succesorii



# Failures may cause incorrect lookup



# Exemplu: Chord - Joins + Stabilization

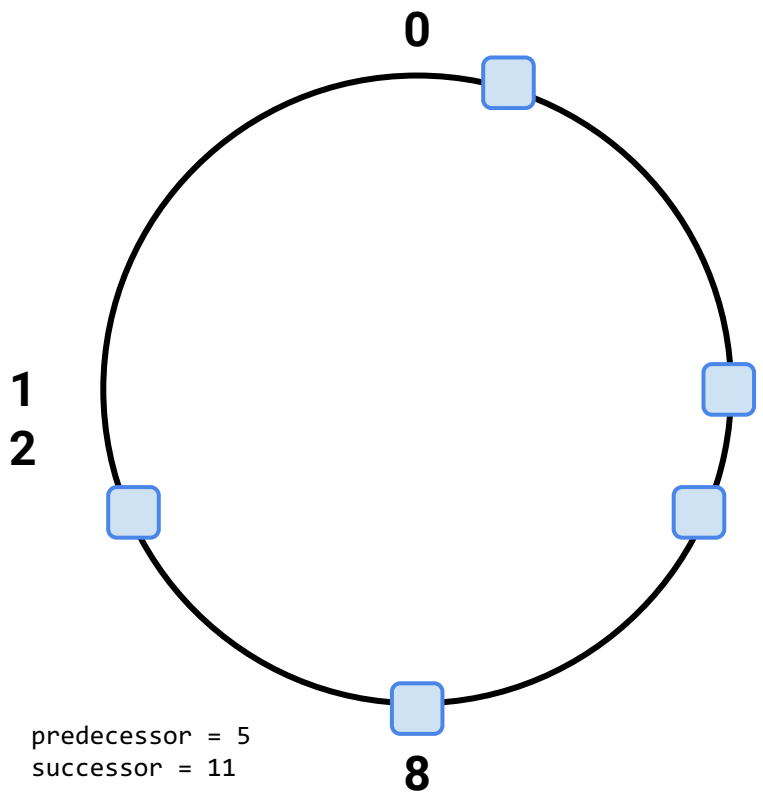



```
join():
  self.predecessor = null
  self.successor = find_successor(self)

stabilize():
  p = self.successor.predecessor
  if p between (self, self.successor):
    self.successor = p
  self.successor.notify(self)

notify(n):
  if self.predecessor == null ||
     n between (self.predecessor, self):
    self.predecessor = n
```

# Chord - Joins + Stabilization

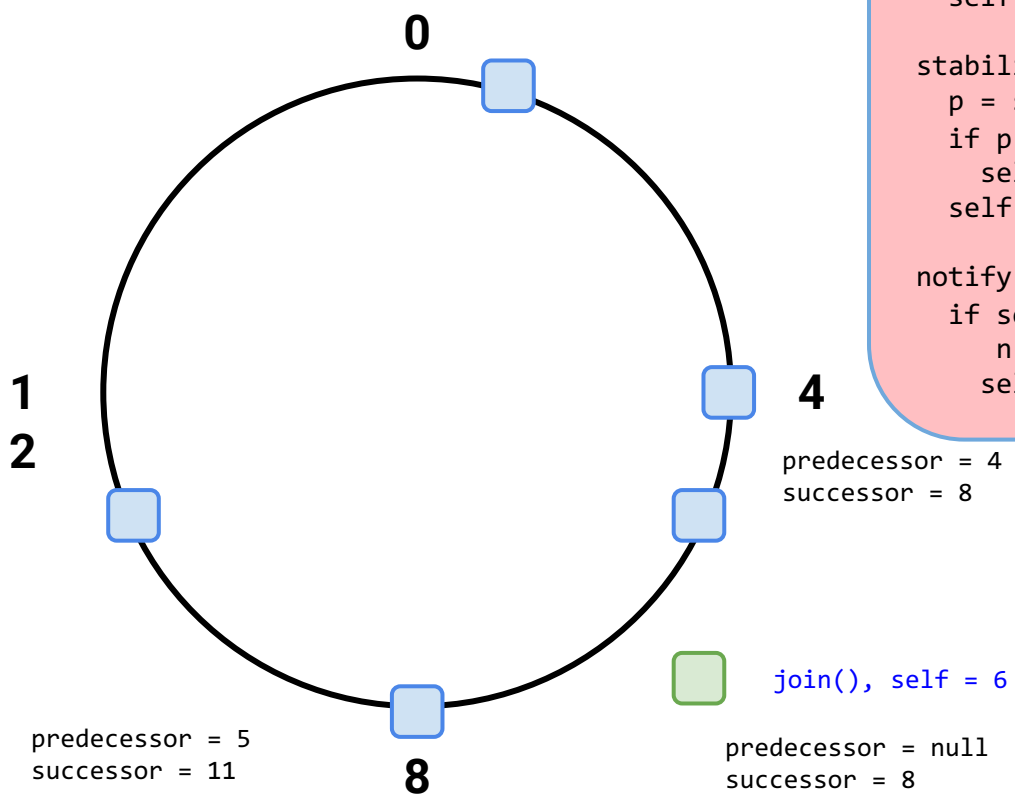



```
join():
  self.predecessor = null
  self.successor = find_successor(self)

stabilize():
  p = self.successor.predecessor
  if p between (self, self.successor):
    self.successor = p
  self.successor.notify(self)

notify(n):
  if self.predecessor == null ||
     n between (self.predecessor, self):
    self.predecessor = n
```

# Chord - Joins + Stabilization

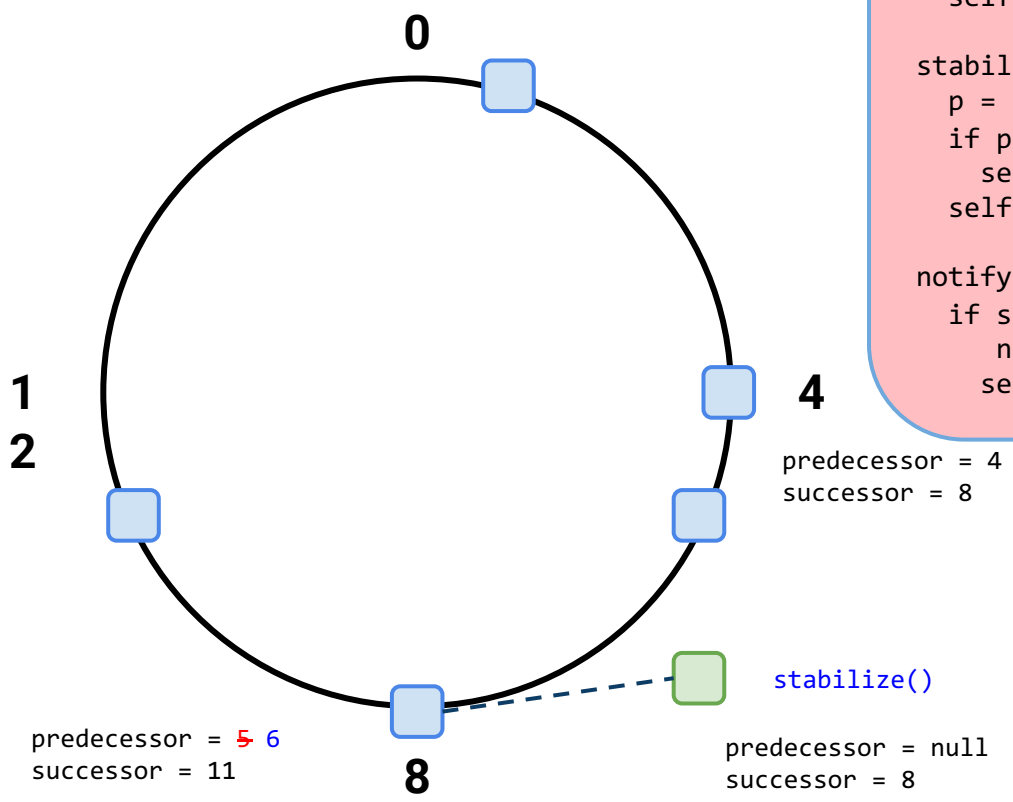



```
join():
  self.predecessor = null
  self.successor = find_successor(self)

stabilize():
  p = self.successor.predecessor
  if p between (self, self.successor):
    self.successor = p
  self.successor.notify(self)

notify(n):
  if self.predecessor == null ||
     n between (self.predecessor, self):
    self.predecessor = n
```

# Chord - Joins + Stabilization




```
join():
  self.predecessor = null
  self.successor = find_successor(self)

stabilize():
  p = self.successor.predecessor
  if p between (self, self.successor):
    self.successor = p
  self.successor.notify(self)

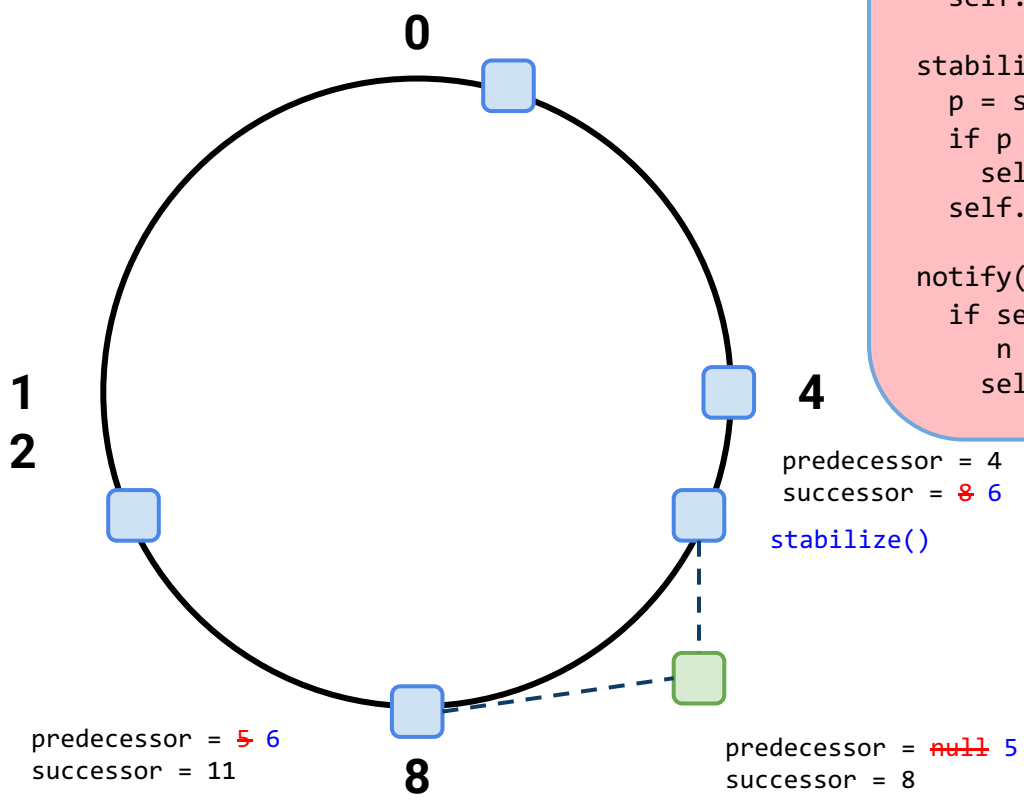
notify(n):
  if self.predecessor == null ||
     n between (self.predecessor, self):
    self.predecessor = n
```

# Chord - Joins + Stabilization

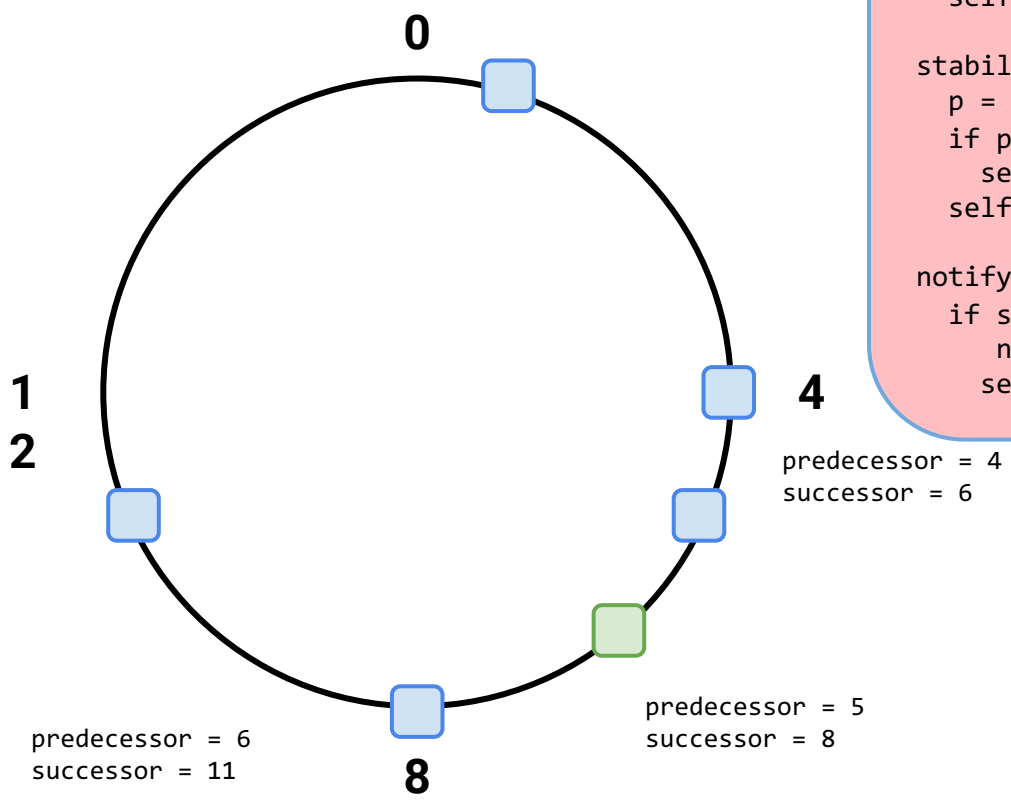
```
join():
  self.predecessor = null
  self.successor = find_successor(self)

stabilize():
  p = self.successor.predecessor
  if p between (self, self.successor):
    self.successor = p
  self.successor.notify(self)

notify(n):
  if self.predecessor == null ||
     n between (self.predecessor, self):
    self.predecessor = n
```



# Chord - Joins + Stabilization

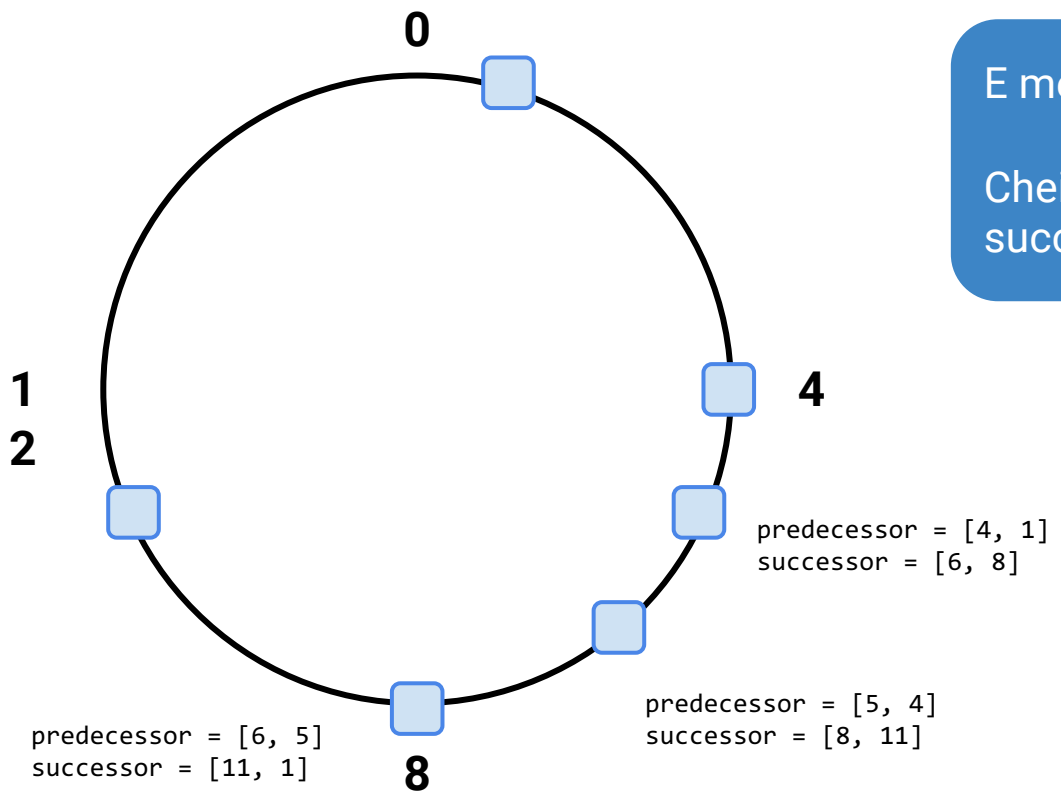



```
join():
  self.predecessor = null
  self.successor = find_successor(self)

stabilize():
  p = self.successor.predecessor
  if p between (self, self.successor):
    self.successor = p
  self.successor.notify(self)

notify(n):
  if self.predecessor == null ||
     n between (self.predecessor, self):
    self.predecessor = n
```

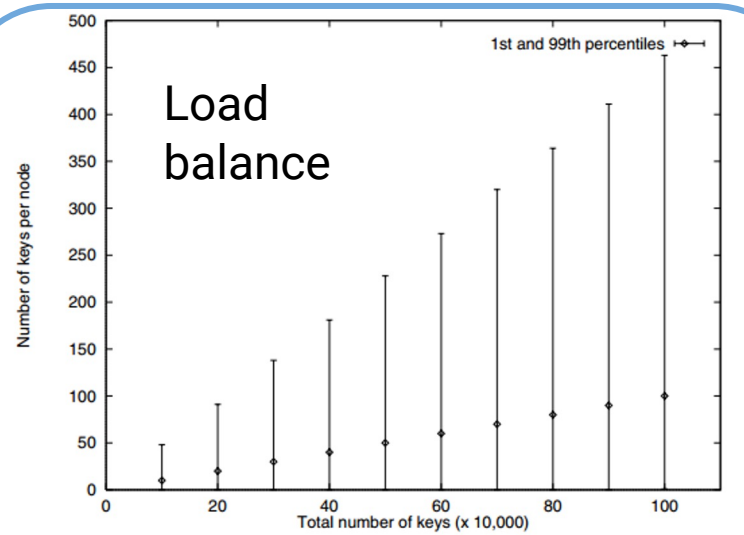
# Chord - Failure + Replication



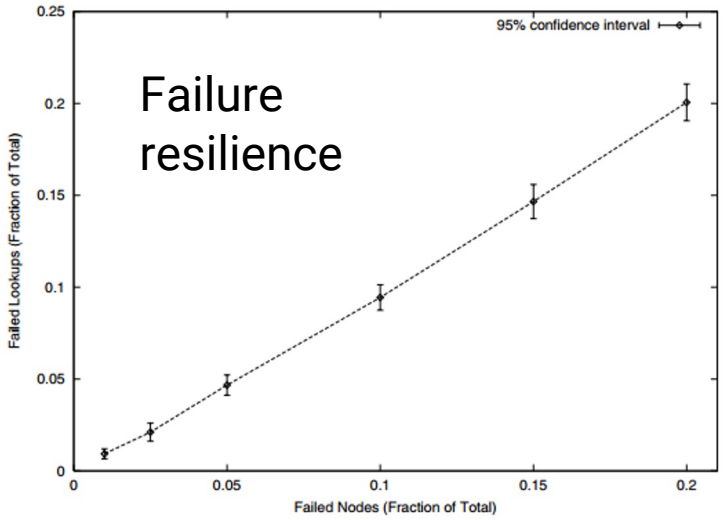

E mentinuta o lista a k succesori

Cheile sunt replicate pe toti k succesorii

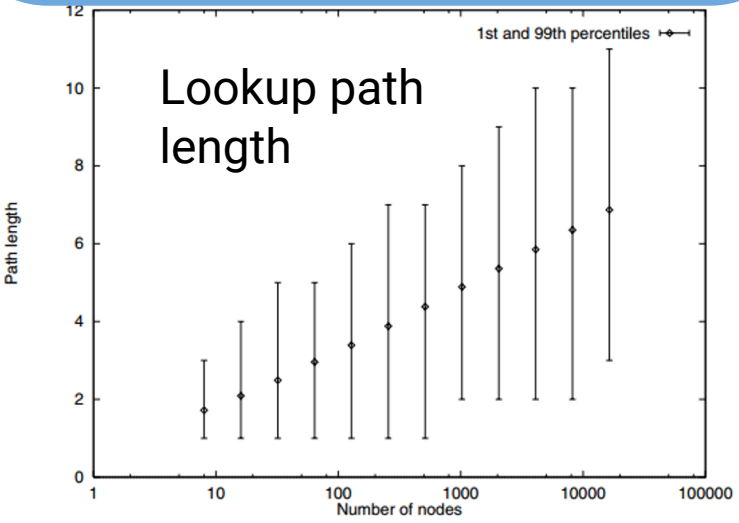
## Load balance



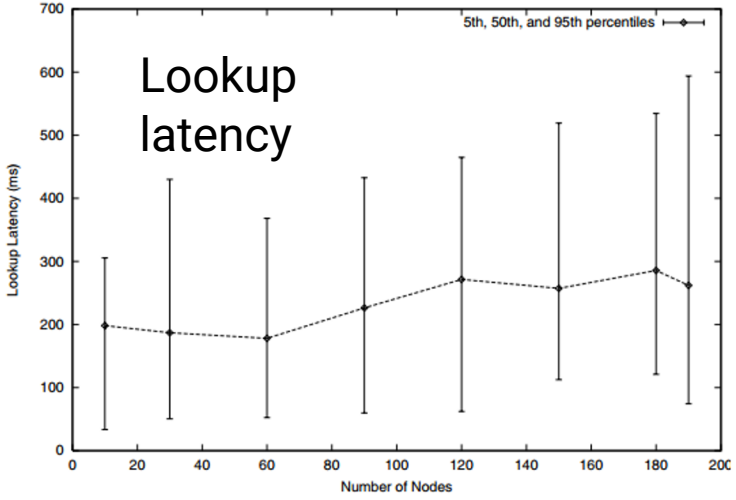

## Failure resilience



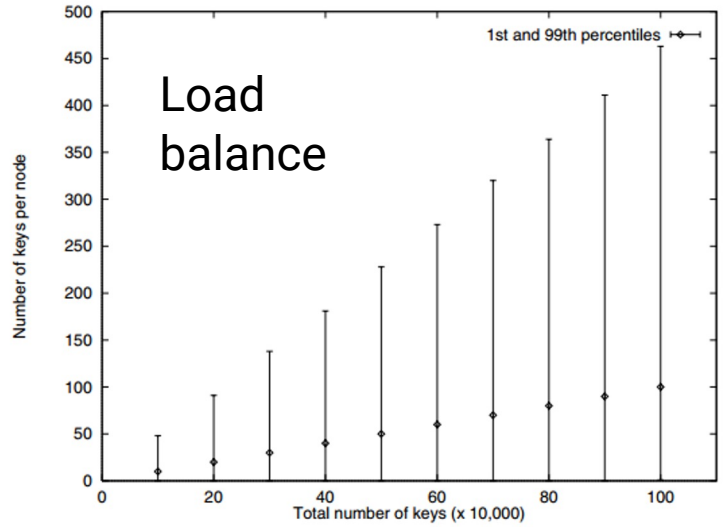

## Lookup path length



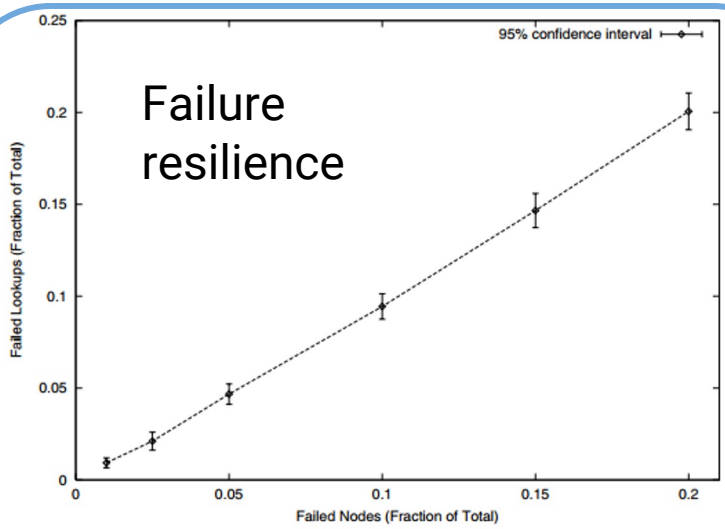

## Lookup latency



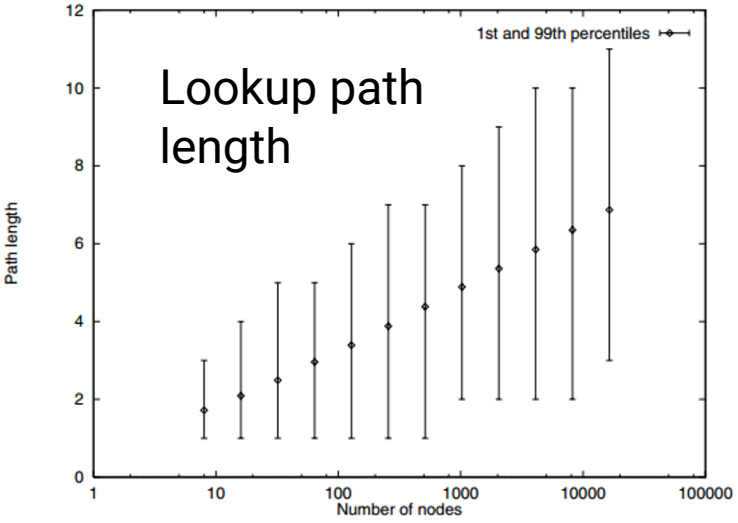
## Load balance



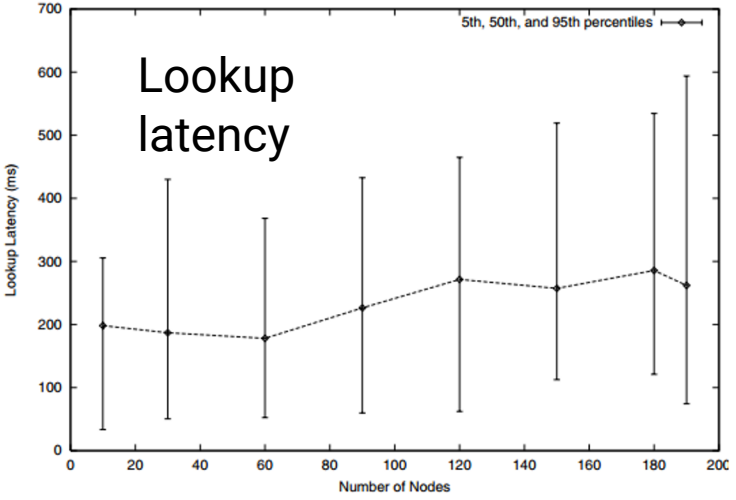

## Failure resilience



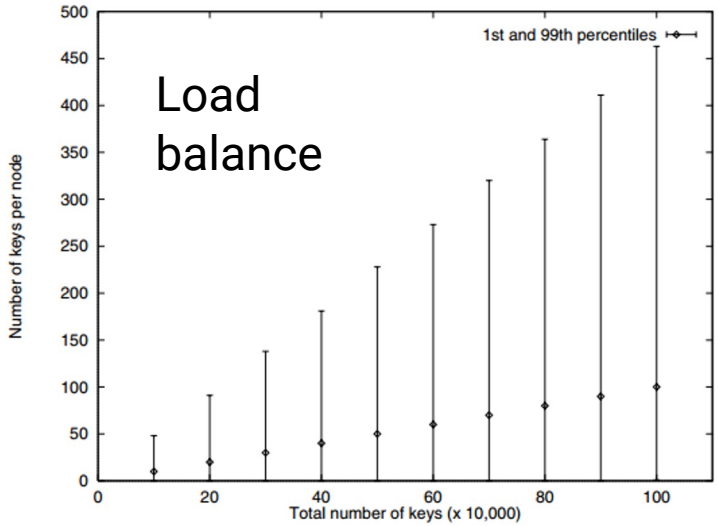

## Lookup path length



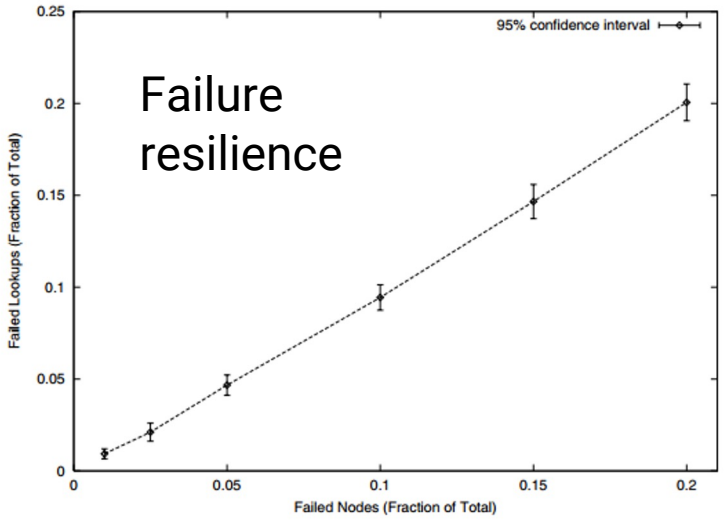

## Lookup latency



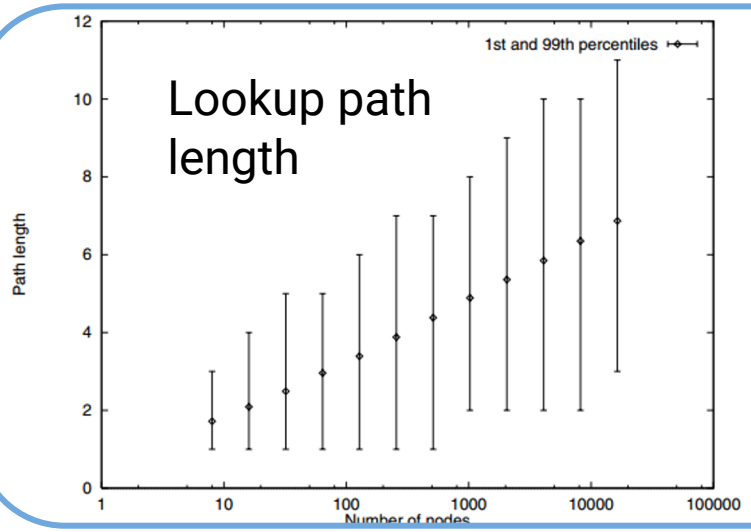
## Load balance



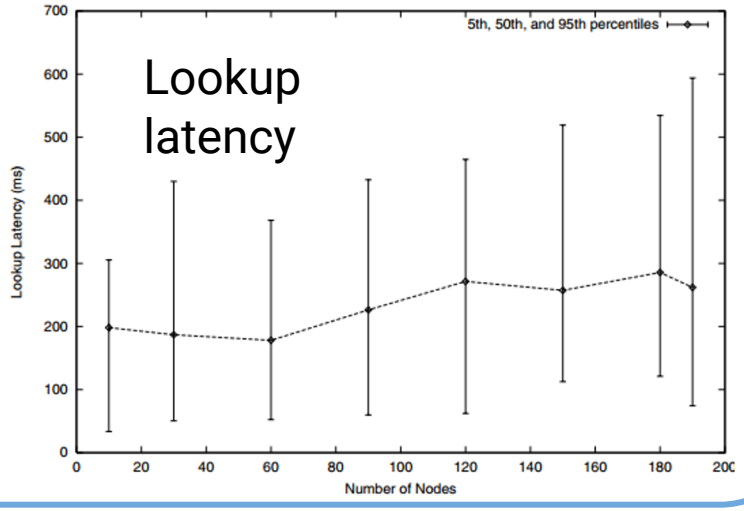

## Failure resilience




## Lookup path length




## Lookup latency



# The Impact of DHT Routing Geometry on Resilience and Proximity

K. Gummadi[†], R. Gummadi[‡], S. Gribble[‡], S. Ratnasamy[§], S. Shenker[¶], I. Stoica[‖] *




## ABSTRACT

The various proposed DHT routing algorithms embody several different underlying routing *geometries*. These geometries include hypercubes, rings, tree-like structures, and butterfly networks. In this paper we focus on how these basic geometric approaches affect the resilience and proximity properties of DHTs. One factor that distinguishes these geometries is the degree of *flexibility* they provide in the selection of neighbors and routes. Flexibility is an important factor in achieving good static resilience and effective proximity neighbor and route selection. Our basic finding is that, despite our initial preference for more complex geometries, the ring geometry allows the greatest flexibility, and hence achieves the best resilience and proximity performance.



## Categories and Subject Descriptors

C.2 [**Computer Systems Organisation**]: Computer Communication Networks

## General Terms

Algorithms, Performance

## Keywords


## 1. INTRODUCTION

The unexpected and unprecedented explosion of peer-to-peer file-sharing, ignited by Napster and refueled by a succession of less legally vulnerable successors (*e.g.*, Gnutella, Kazaa), inspired the development of distributed hash tables (DHTs). DHTs offer a promising combination of extreme scalability (scaling typically as $\log n$) and useful semantics (supporting a hash-table-like `lookup` interface), and have been proposed as a substrate for many large-scale distributed applications (see, for example, [5, 12, 22]).

Our focus here is not on the uses of DHTs but on the underlying DHT routing algorithms. A wide variety of DHT routing algorithms have been proposed, and the list grows longer with each passing conference. However, the DHT routing literature is still very much in its infancy; as a result, most DHT routing papers describe new algorithms and very few provide more general insight *across* algorithms or usefully compare *between* algorithms.

In this paper we attempt to take a very small step in this direction by looking at the basic *geometry* underlying DHT routing algorithms and how it impacts their performance in two important areas: static resilience and proximity routing. While the current collection of DHT routing algorithms differs in many respects, perhaps the most fundamental distinction between them lies in their different routing geometries. For instance, CAN [19] (when the dimension is taken to be

Gummadi, K., Gummadi, R., Gribble, S., Ratnasamy, S., Shenker, S., & Stoica, I. (2003, August). The impact of DHT routing geometry on resilience and proximity. In *Proceedings of the 2003 conference on Applications, technologies, architectures, and protocols for computer communications* (pp. 381-394). ACM.

# Topologii de rutare in spatele unor DHT

Ring (Chord)
Tree (Tapestry, PRR)
Hypercube (CAN)
Butterfly (Viceroy)
XOR (Kademlia)
Hybrid (Pastry, Bamboo)

# Proximity + Resilience

Proximity – Alegerea unor rute prin vecini “geografic apropiati” reduce latenta

Resilience – Continuarea rutarii unor cereri chiar cand apar partitionari si defecte la nivelul retelei

# Flexibility

Neighbor selection – optiunea de a selecta ce noduri peer sa pastram in tabela de rutare

Route selection – optiunea de a selecta unde sa rutam o anumita destinatie

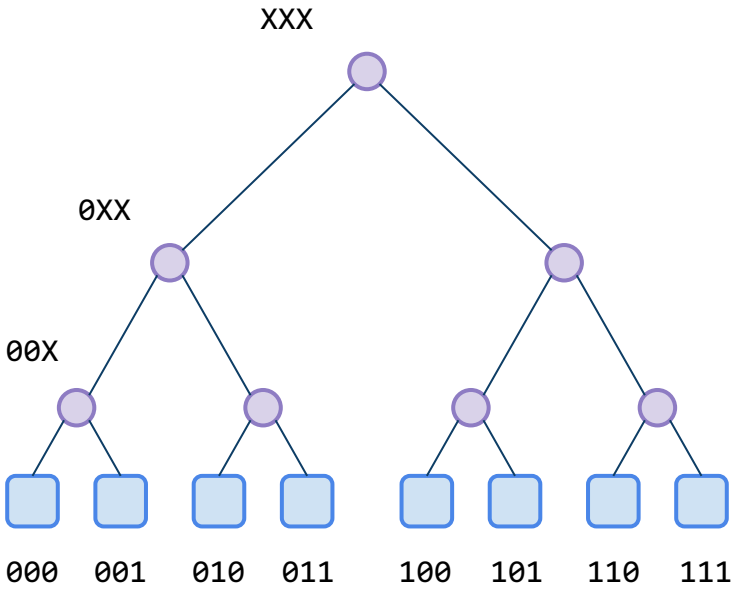
Flexibility in neighbor selection -> good proximity

Flexibility in route selection -> good resilience

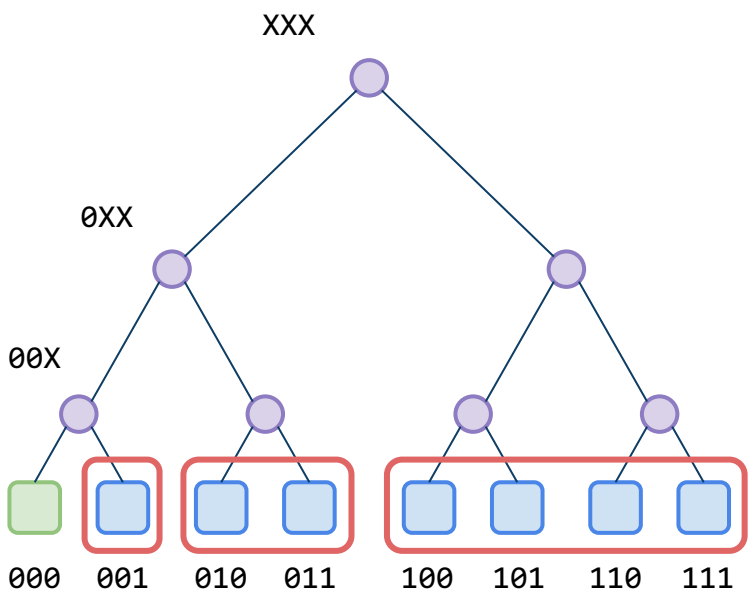
# Topologii de rutare in spatele unor DHT

**Ring** (Chord)
**Tree** (Tapestry, PRR)
**Hypercube** (CAN)
Butterfly (Viceroy)
XOR (Kademlia)
Hybrid (Pastry, Bamboo)

# Topologii de rutare in spatele unor DHT - Arbore

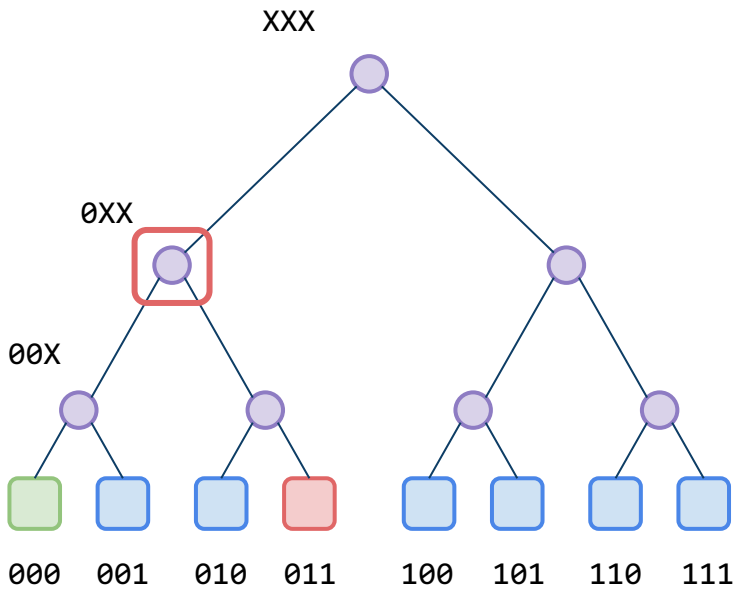


# Topologii de rutare in spatele unor DHT - Arbore




Neighbor selection – un nod pentru fiecare prefix din subarborele opus

# Topologii de rutare in spatele unor DHT - Arbore



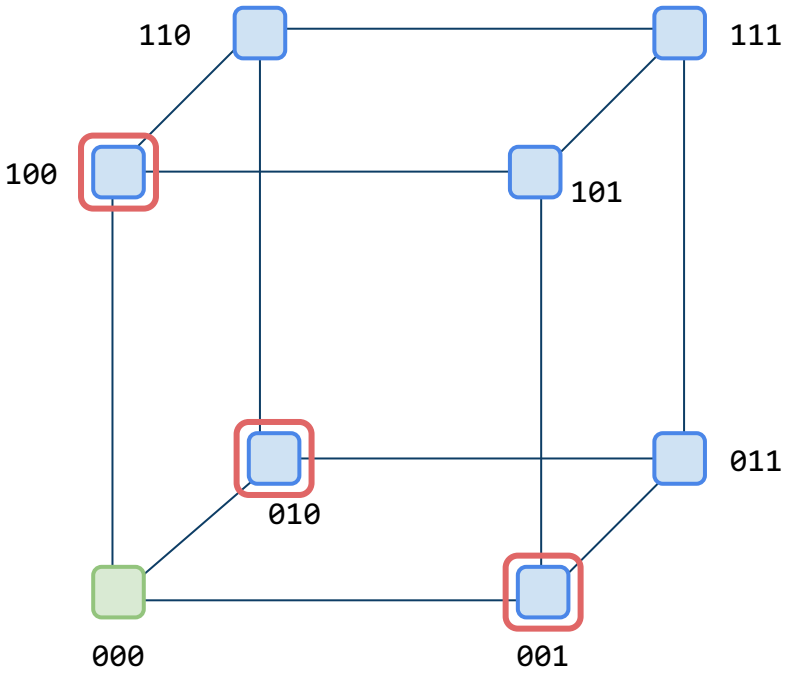

Neighbor selection – un nod pentru fiecare prefix din subarborele opus

Route selection – rutarea catre vecinul din subarborele destinatiei

Good flexibility in neighbor selection

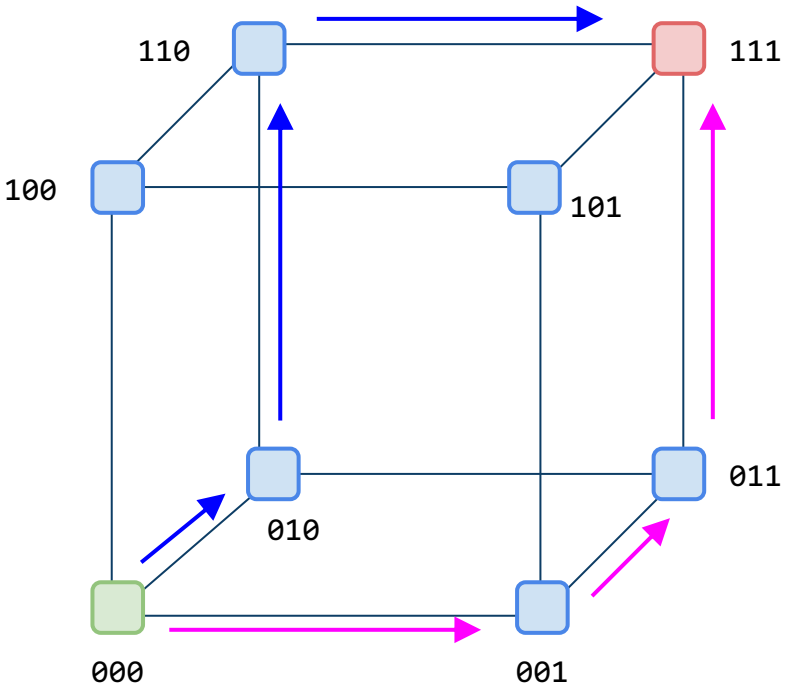
Poor flexibility in route selection

# Topologii de rutare in spatele unor DHT - Hipercub




Neighbor selection – vecinii difera printr-un bit in descompunerea binara

# Topologii de rutare in spatele unor DHT - Hipercub



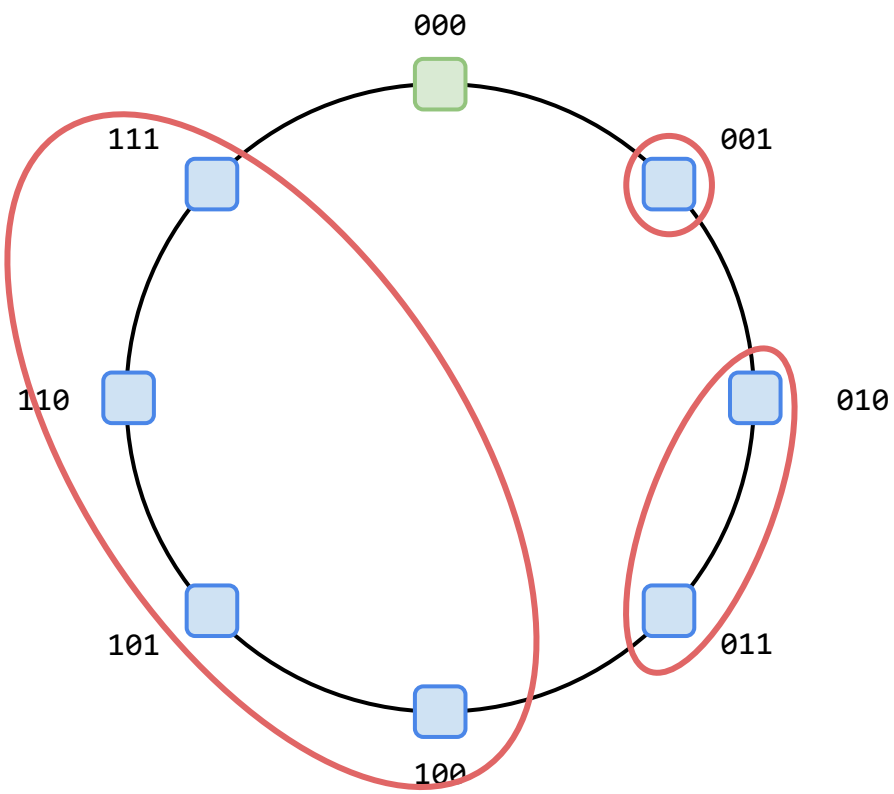

Neighbor selection – vecinii difera printr-un bit in descompunerea binara

Route selection – rutare catre destinatie prin corectarea oricarui bit diferit

Poor flexibility in neighbor selection

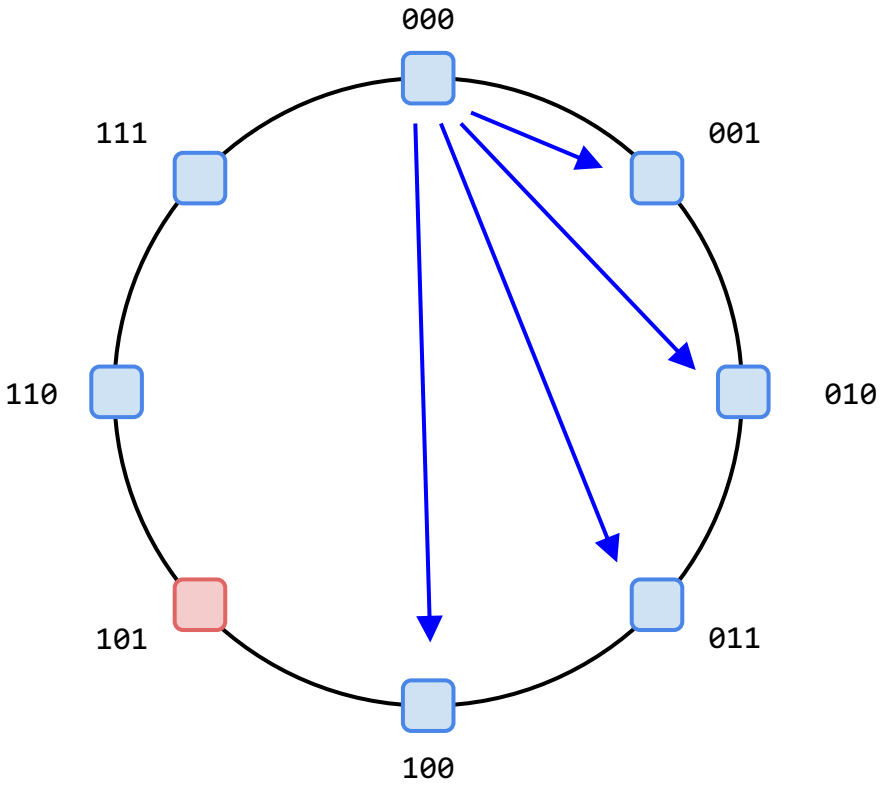
Good flexibility in route selection

# Topologii de rutare in spatele unor DHT - Inel




Neighbor selection – un vecin in fiecare interval de finger

# Topologii de rutare in spatele unor DHT - Inel




Neighbor selection – un vecin in fiecare interval de finger

Route selection – rutarea catre destinatie prin progresarea de-a lungul inelului

Good flexibility in neighbor selection

Good flexibility in route selection

# BitTorrent

- Principalul exemplu de sistem P2P in folosinta astazi (alaturi de Boinc):
  - Dezvoltat de Cohen in ‘01
  - Mare parte din traficul de Internet astazi!
  - Folosit pentru transfer de continut legal si mai putin legal (aici intervine rolul DHT)
- Date livrate folosind “torrents”:
  - Fisierele sunt transferate in bucati, e favorizat paralelismul
  - Foloseste un tracker sau un decentralized index (DHT)

Bram Cohen (1975 -)



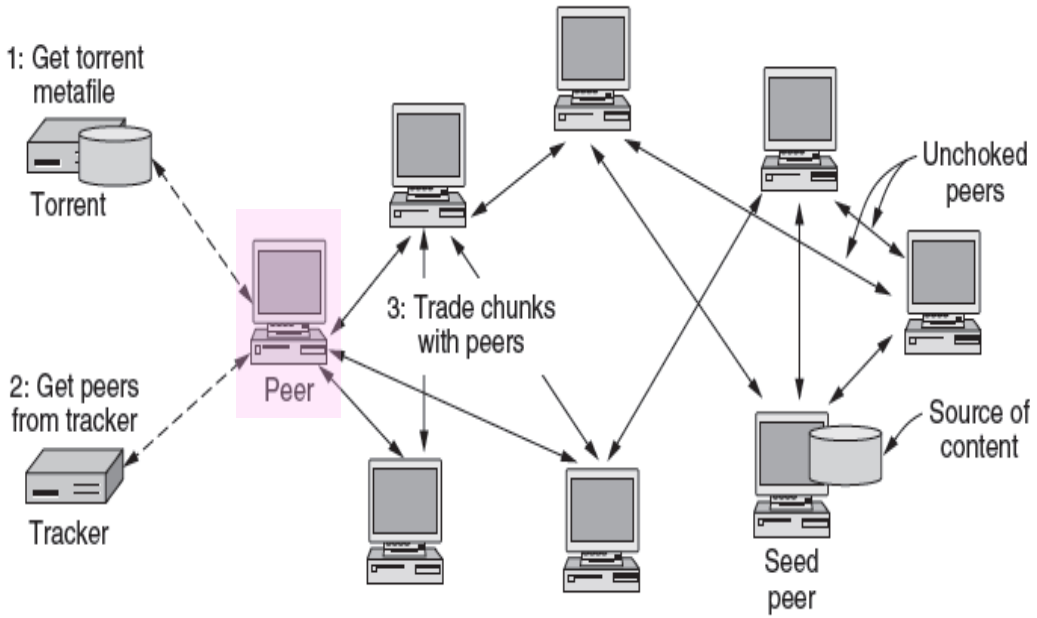
# Exemplu: BitTorrent

1. User clicks on download link
   - Gets ***torrent*** file with content hash, IP addr of ***tracker***
2. User's BitTorrent (BT) client talks to tracker
   - Tracker tells it **list of peers** who have file
3. User's BT client downloads file from one or more peers
4. User's BT client tells tracker it has a copy now, too
5. User's BT client serves the file to others for a while

Provides huge download bandwidth, without expensive server or network links

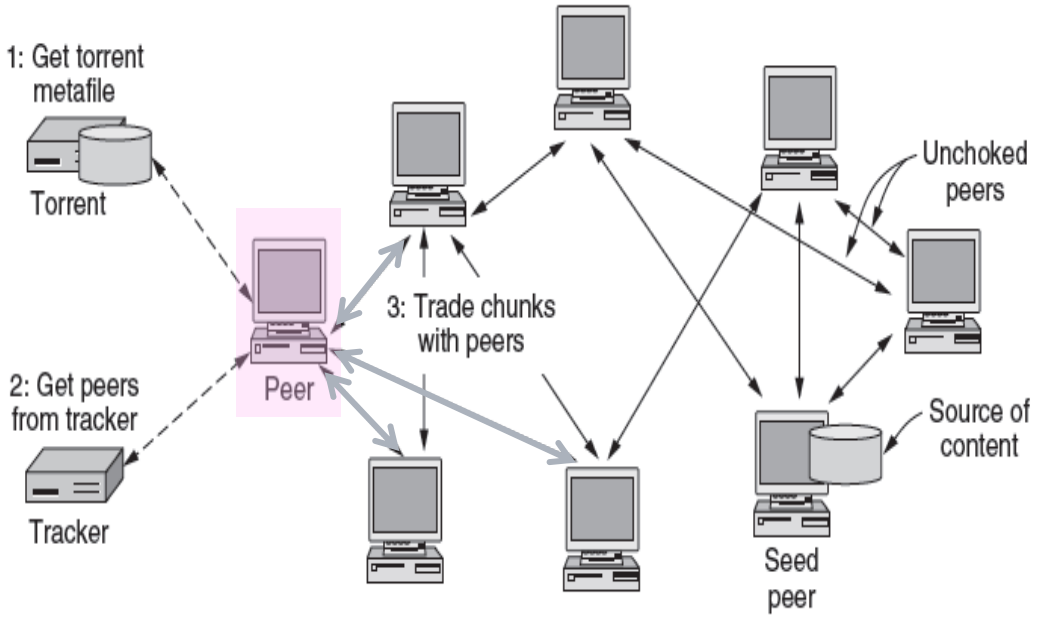
# BitTorrent Protocol

- Toti peers (cu exceptia celor seed) regasesc torrenti in acelasi timp



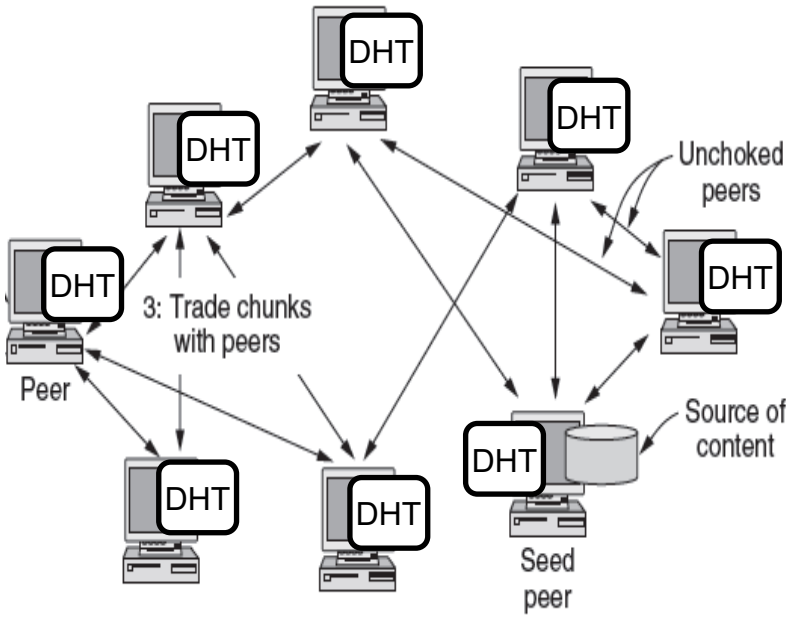
# BitTorrent Protocol (2)

- Fisierele sunt impartite in bucati, ceea ce permite descarcarea paralela (viteza de download)

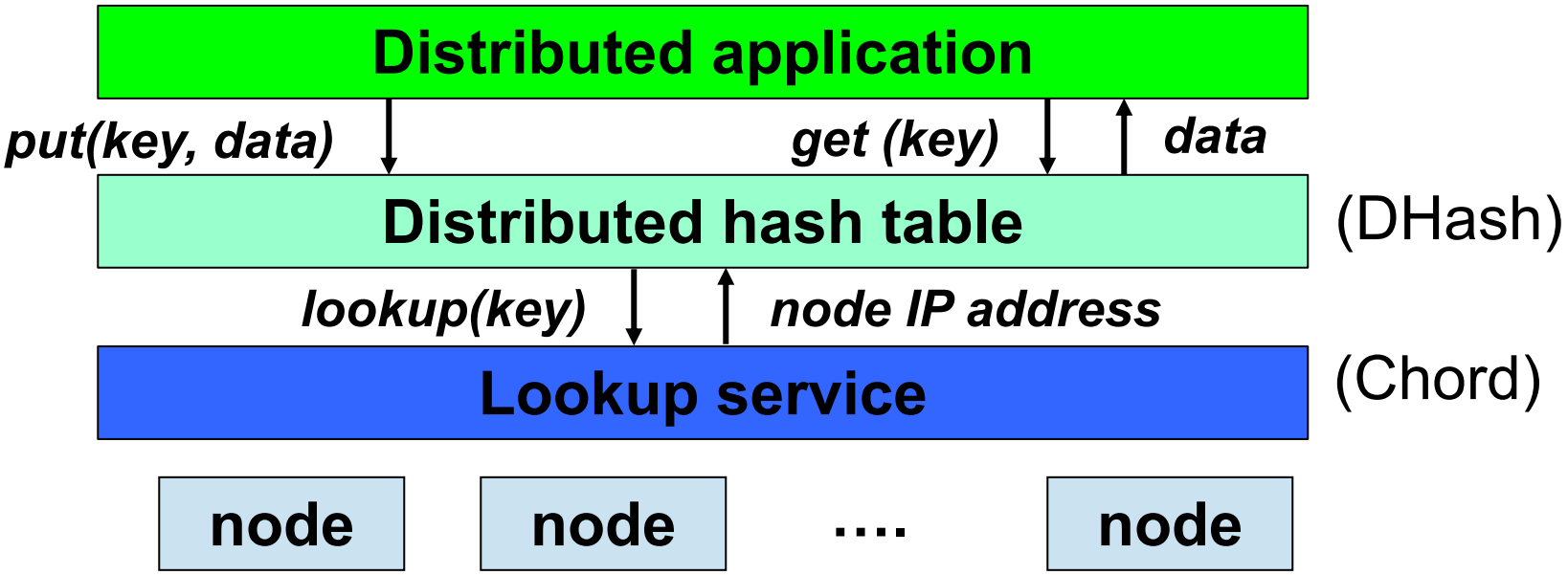


# BitTorrent Protocol (3)

- Indexul DHT (distribuit intre peers) este complet descentralizat



# Cooperative storage with a DHT




- Aplicatiile pot fi **distribuite** pe multe noduri
- DHT **distribuie stocarea de date** pe mai multe noduri

# BitTorrent si DHT

- BitTorrent poate folosi DHT in loc de (sau lucreaza impreuna cu un) tracker

- Clientii BT folosesc DHT:
  - Key = **file content hash** (“infohash”)
  - Value = **IP address of peer** willing to serve file

- Clientul:
  - `get(infohash)` pentru a gasi alti client ce pot deserve o descarcare
  - `put(infohash, my-ipaddr)` pentru a se identifica ca furnizor de content

# La ce foloseste un DHT pentru BitTorrent?

- DHT e un tracker in sine, mai putin fragmentat decat alti tracker
  - Nodurile peer se pot regasi unele cu altele mai usor, chiar si in prezenta defectelor
- In plus, un tracker classic este expus unor probleme de **legalitate**

# Sumar

- Sisteme peer-to-peer
- Distributed Hash Tables
  - Chord
- Topologii din spatele DHT
- Exemplificare BitTorrent